



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

COURSE CODE: EEE 4308

COURSE NAME: DIGITAL ELECTRONICS LAB

PROJECT: SAP-1 Architecture based 8-bit Computer Design and Implementation.

Final Report

GROUP NAME: Boolean Squad

GROUP MEMBERS:

1. IKRAM HOSSAIN (230021111)
2. NADMAN IBNE MAMUN (230021114)
3. FARHAN ISHRAQ (230021124)
4. SAMIN YASAR SAMI (230021138)
5. ZAVID ATAHAR SAMIN (230021155)
6. AGHA MAKINUN AMIN CHOWDHURY (230021232)
7. RAZEEN ABRAR RAHMAN (230021215)

Supervisor Name:

FATEMA TASNIM OYSHI

Junior Lecturer,

Department of EEE, IUT

CLOCK PULSE GENERATOR

The clock pulse generator is the timing center of the computer. It produces a continuous stream of square waves that synchronize all operations within the system.

Operation Modes:

1. **MANUAL:** It allows for single-stepping through instructions using a push button.
2. **AUTO:** Generates a steady stream frequency to execute programs at a set speed.

Function: It provides CLK signals to the registers, the program counter and the control-sequencer to ensure data is moving at correct intervals.

PROGRAM COUNTER: The program counter is a 4-bit counter that tracks the memory address of the next instruction to be fetched. It counts from 00002 to 11112 (0 to 15 in decimal).

Control Signals:

1. **CE:** Increments the program counter on the next clock signal.
2. **CO:** It puts the current counter value on the bus.
3. **RESET:** It resets the counter to 0000.
4. **CLK:** It is the clock pulse input.

Function: It ensures that the computer executes instructions in the correct sequence.

BOOLEAN EXPRESSION:

The Boolean expression derived from this truth table is given below:

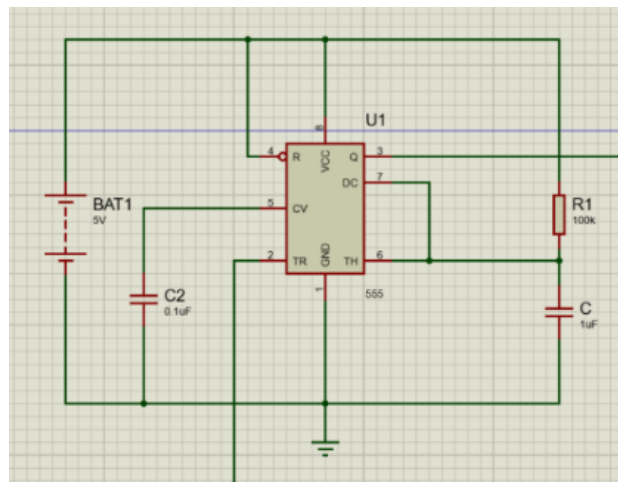
$$Y = \bar{H} \cdot ((\bar{S} \cdot M) + (S \cdot A))$$

TRUTH TABLE:

MODE (S)	Halt (H)	Auto Pulse (A)	Manual Pulse (M)	Output (Y)
X	1	X	X	0
1 (auto)	0	0	X	0
1 (auto)	0	1	X	1
0 (manual)	0	X	0	0
0 (manual)	0	X	1	1

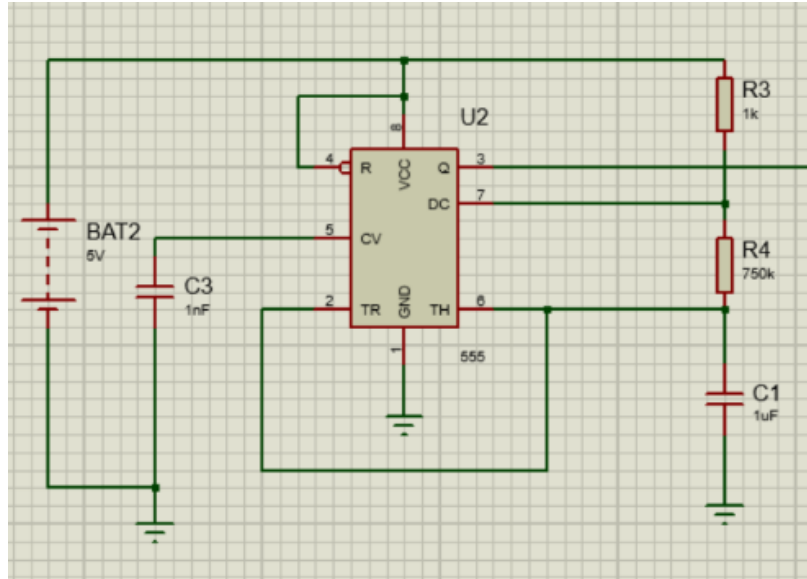
Monostable Multivibrator using 555 Timer IC:

The circuit given in the right side is responsible for the manual mode of our clock pulse generator. Here the Pin2 or the trigger pin is connected to a triggering circuit which is described later.



Monostable Multivibrator using 555 Timer IC:

The circuit given in the right side is responsible for the auto mode of our clock pulse generator.

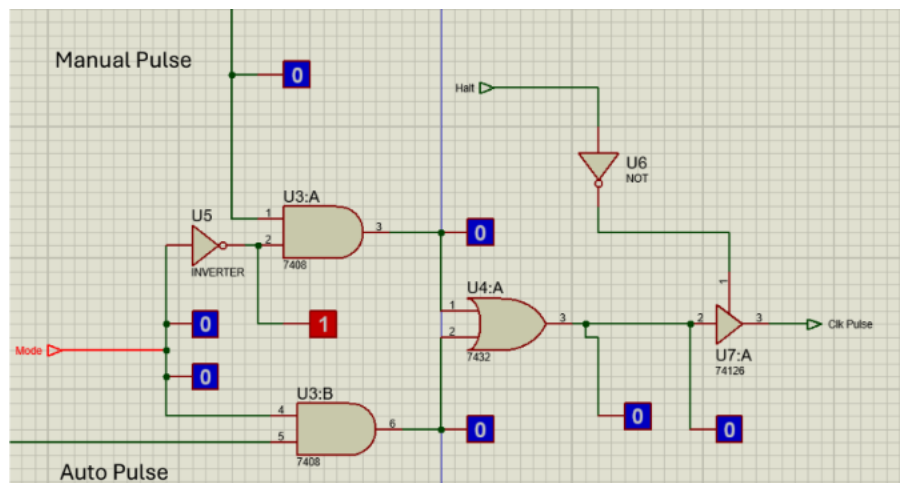


Mode Selection Circuit:

Mode=0 Manual pulse gets through

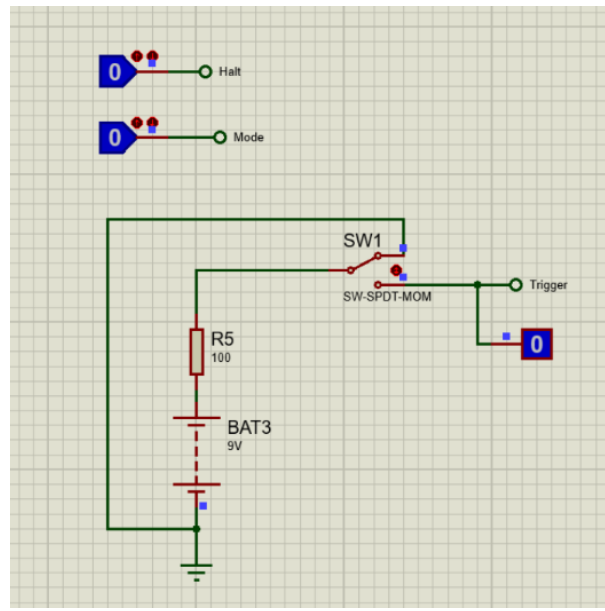
Mode=1 Auto pulse gets

through



Triggering Circuit and Control Center:

Our triggering method consists of a simple circuit with a SPDT momentary switch that changes its position for a moment which results in a flow of current in the pin2 of our manual pulse generator or mono-stable multivibrator circuit. Again there is Halt to stop the clock pulse and Mode button to control the mode of operation. It should be mentioned that Halt=1 indicates the pulse stops, Mode=0 indicates Manual pulse and Mode=1 indicates Auto pulse.



Overall Circuit Design:

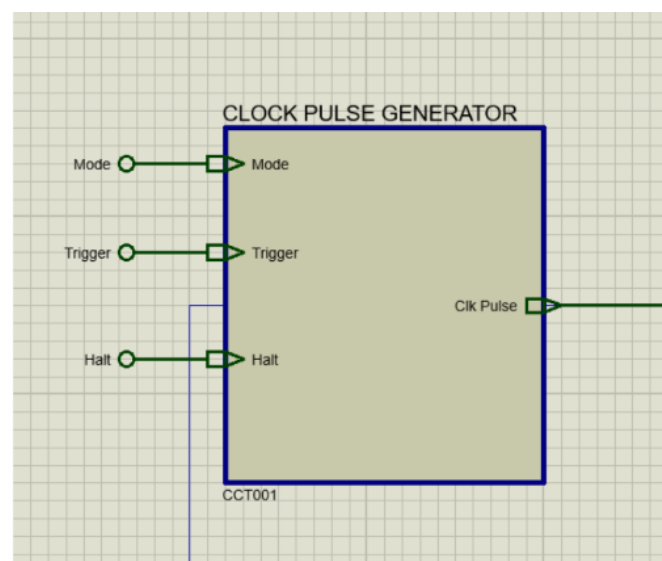


Fig. Subcircuit diagram of Clock Pulse Generator

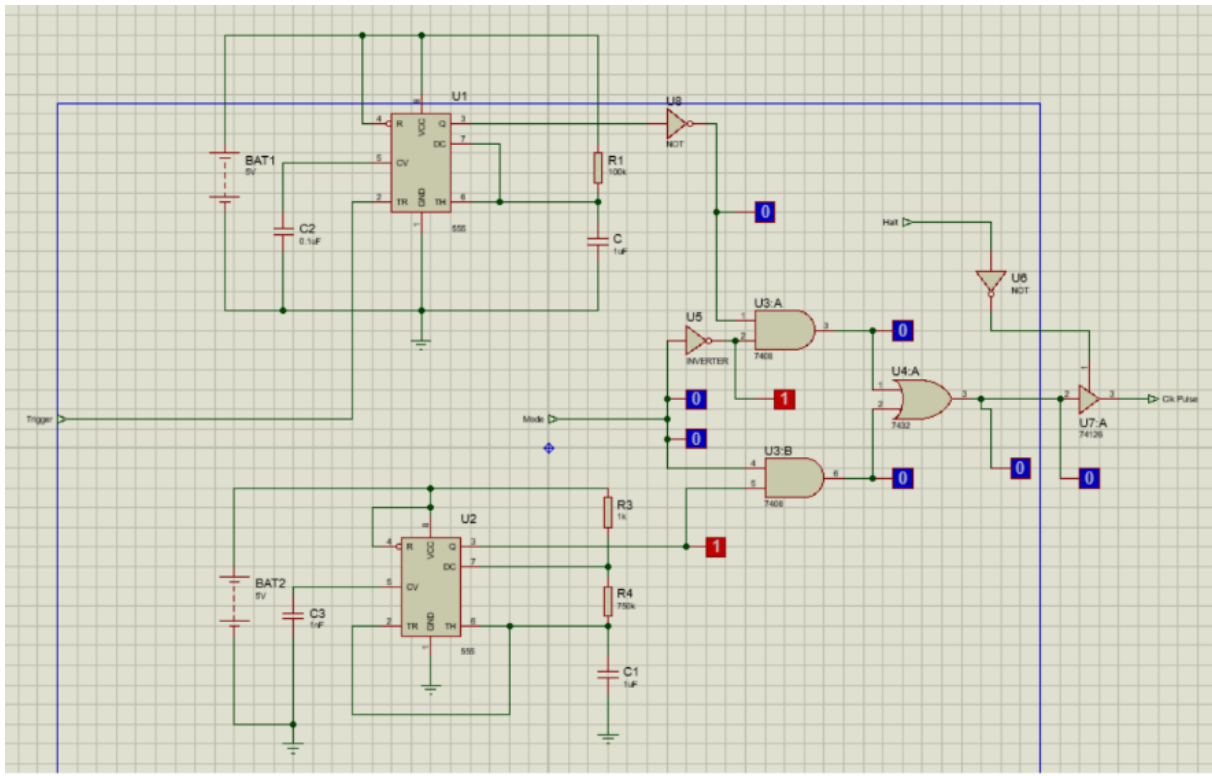
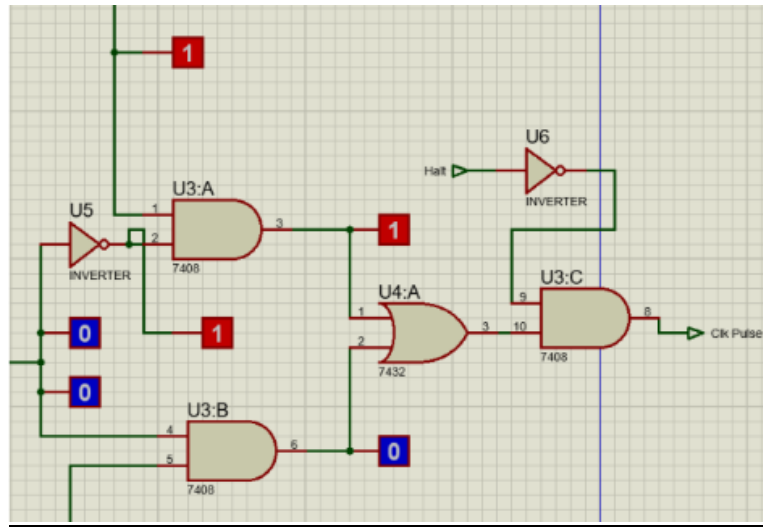


Fig. breadboard diagram of Clock Pulse Generator

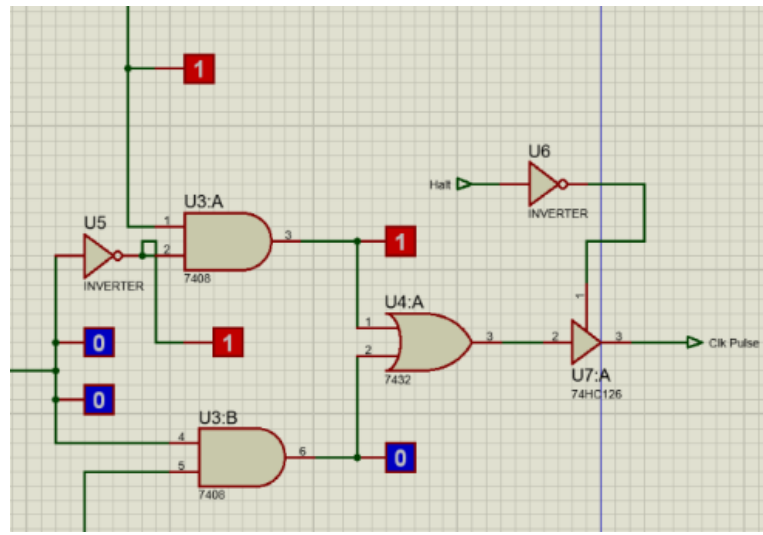
Equipment / Component List:

- 1. OR Gate IC(74HC32), AND Gate IC (74HC08), NOT Gate IC (74HC04):** Used in the selection circuit.
- 2. Tri-State Buffer (74126):** Used to interface the programme counter with the system bus and prevent bus contention.
- 3. 555 Timer IC :** Used as the main pulse generator
- 4. Resistors**
- 5. Capacitor (non electrolytic)**
- 6. Battery (5V DC):** Required to power ICs used in the circuit.
- 7.Connecting Wires:** Used to interconnect all components in the circuit.

Problems Faced:



Initial



Final

Due to our Mode selection circuit, when And Gate IC was used to Halt the Pulse there were some unwanted pulse seen in the output which could result in unwanted output for example the Program Counter connected to it could increment its value. To solve the problem, we just changed the And Gate IC with Tri state buffer (74HC1256).

Discussion:

Outcomes:

The overall final design of the Clock Pulse Generator using 555 Timer IC can be considered as a reliable pulse generator for our SAP1 8Bit computer.

Limitations:

Even though the Clock Pulse generator looks easy on a software simulation there are many factors to be considered in case of hardware implementation. And similarly, despite its improved performance, the designed programme counter has some limitations. The use of discrete logic ICs increases the overall hardware complexity and wiring, which may become inefficient for larger bit-width counters. The usage of 555timer can result in not being able to generate High or Precise Frequency. The counter also operates only in an increment mode and does not support advanced operations such as loading a jump address or decrementing. Additionally, the design relies on a fixed clock frequency and does not include power-saving or dynamic clock control features. These limitations make the current design suitable mainly for educational and basic computer architectures like SAP-1.

Future Development

Several improvements can be made to enhance the functionality of the Clock Pulse Generator and Programme Counter in future versions. In case of Pulse Generator, A parallel load feature can be added to support jump and branch instructions, making the system more flexible. The counter can also be redesigned using integrated synchronous counter ICs or programmable logic devices such as CPLDs or FPGAs to reduce hardware complexity. Adding clock gating or enable-based power control can improve efficiency. Furthermore, expanding the counter width and integrating it with a more advanced control unit would allow

this design to be used in more complex computer architectures beyond SAP-1.

PROGRAM COUNTER

In this project, an 8-bit synchronous program counter has been designed for the SAP-1 computer to generate sequential instruction addresses in a reliable and controlled manner. Unlike an asynchronous counter, all JK flip-flops in this design receive the same clock signal, which ensures that every bit of the program counter updates at the same instant and eliminates ripple delay and temporary invalid states on the address bus. The JK flip-flops are operated in toggle mode, and AND gates are used to decide when each higher-order bit should toggle by checking whether all lower-order bits are high, allowing proper binary counting while remaining fully synchronous. A reset circuit is included to initialize the program counter to zero at the start of operation, ensuring correct program execution from the first instruction. The outputs of the program counter are connected to a tri-state buffer (74LS245), which allows the counter to place its value onto the system bus only when enabled, preventing bus contention with other components. Overall, this synchronous program counter provides stable, glitch-free address generation and integrates effectively with the SAP-1 control and bus structure.

TRUTH TABLE:

RESET	CE	CO	CLK	OUTPUT (Q3 Q2 Q1 Q0)	MODE
1	X	X	X	0 0 0 0	Reset
0	0	1	↑	No change	Hold
0	1	1	↑	Count +1	Increment
0	X	0	X	0 0 0 0	disconnect

BOOLEAN EXPRESSION:

The output expressions for the 4 bits are given below:

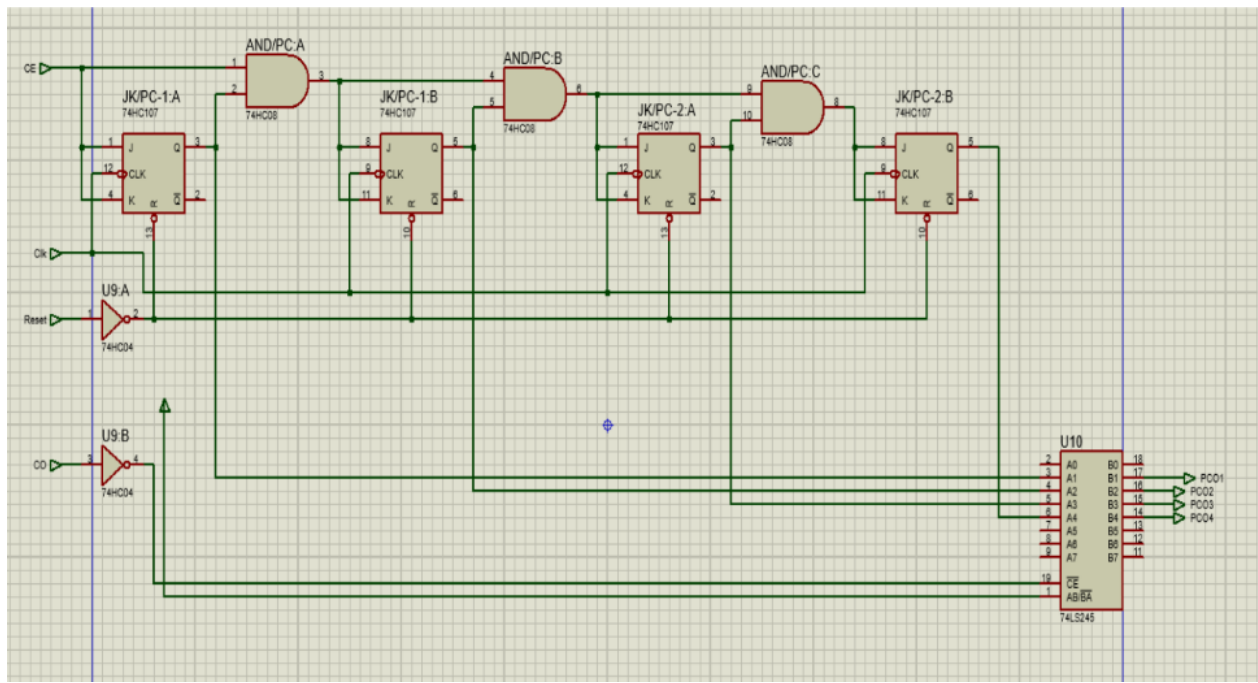
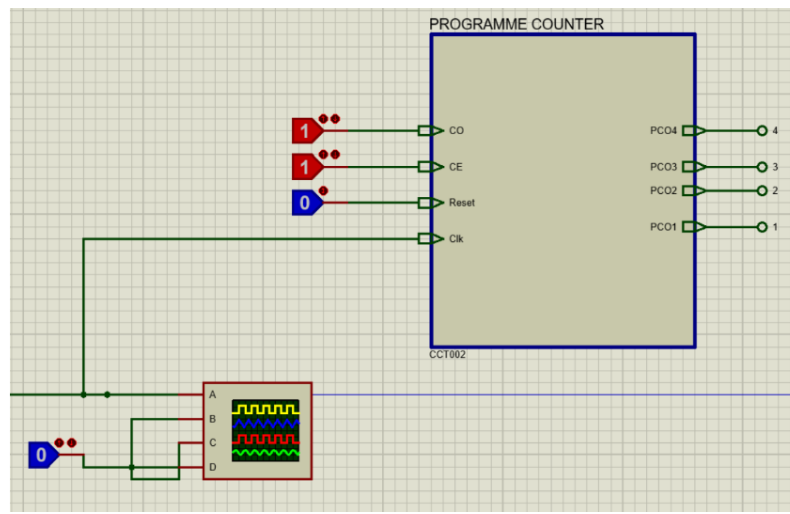
$$Q0_next = \bar{R} (CE \oplus Q0)$$

$$Q1_next = \bar{R} ((CE \cdot Q0) \oplus Q1)$$

$$Q2_next = \bar{R} ((CE \cdot Q0 \cdot Q1) \oplus Q2)$$

$$Q3_next = \bar{R} ((CE \cdot Q0 \cdot Q1 \cdot Q2) \oplus Q3)$$

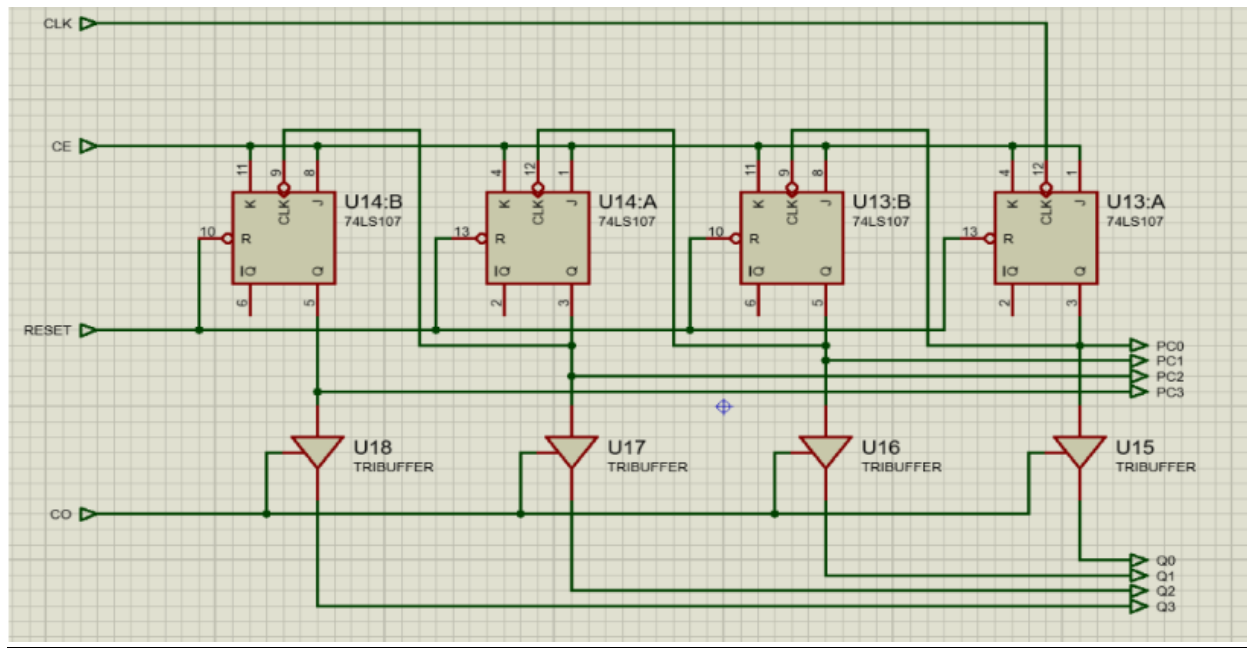
Circuit Diagram:



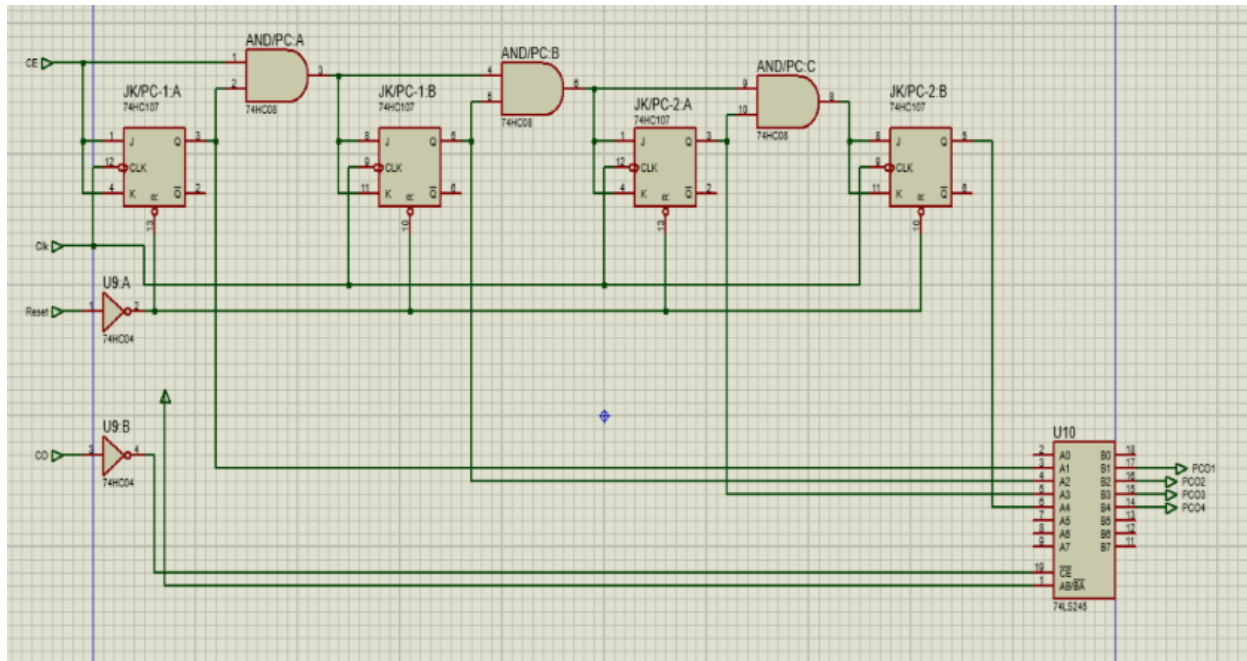
Equipment / Component List

- 1. JK Flip-Flop IC (74HC107):** Used to store and toggle the binary count of the programme counter in synchronous mode.
- 2. AND Gate IC (74HC08):** Used to generate the required toggle conditions for higher-order JK flip-flops in synchronous counting.
- 3. NOT Gate IC (74HC04):** Used for signal inversion, mainly for reset and control signal conditioning.
- 4. Tri-State Buffer IC (74LS245):** Used to interface the programme counter with the system bus and prevent bus contention.
- 5. Clock Pulse Generator:** Provides the common clock signal required for synchronous operation of all flip-flops.
- 6. Reset Switch / Reset Signal Source:** Used to initialize the programme counter to zero.
- 7. Control Signals (CE, CO, AB/BA):** Used to enable counting and control data flow from the programme counter to the bus.
- 8. Power Supply (5V DC):** Required to power all TTL and CMOS logic ICs used in the circuit.
- 9. Connecting Wires:** Used to interconnect all components in the circuit.

Problems Faced:



Intial



Final

At the initial stage of the programme counter design, an asynchronous counter was used along with four individual tri-state buffers to place the counter output onto the bus. This approach created timing instability because the counter outputs did not change simultaneously, which could lead to temporary incorrect address values on the bus. Additionally, proper signal conditioning was not applied at first, as NOT gates were not used for the reset and enable control signals. Due to this, the programme counter did not reset reliably and sometimes responded incorrectly to control inputs. These issues highlighted the limitations of the asynchronous design for a bus-based system like SAP-1. To solve these problems, the design was modified by implementing a synchronous counter with proper AND-gate controlled toggle logic and correct use of NOT gates for reset and enable signals, resulting in stable, predictable, and glitch-free operation.

Discussion:

Outcomes

Final implementation of the programme counter using a synchronous design resulted in stable and reliable address generation for the SAP-1 computer. Compared to the earlier asynchronous approach, the synchronous counter eliminated ripple delay and prevented temporary invalid outputs on the address bus. All flip-flops updated simultaneously using a common clock, which ensured correct instruction sequencing. The use of standard logic ICs such as JK flip-flops, AND gates, and a tri-state buffer improved modularity and made the design easier to understand and debug. Overall, the synchronous programme counter successfully met the timing and reliability requirements of a bus-based SAP-1 architecture.

Registers

Registers are small, fast memory units inside a CPU. They temporarily store data that the CPU needs immediately for processing. Registers help the CPU work faster because accessing data from registers is much quicker than accessing it from main memory.

Accumulator/ A Register (AC)

The Accumulator or A register is the primary working register in a CPU. It is used to store data temporarily and is the main place where the Arithmetic Logic Unit (ALU) performs operations.

- During a calculation, the ALU usually takes one input from the AC.
- The result of any arithmetic or logic operation is stored back in the AC, making it the main storage for immediate results.
- The AC can also send data to the bus (for memory or I/O communication) or receive data from the bus, depending on the instruction being executed.

In simple terms, the AC is like the CPU's main calculator, holding the numbers you are currently working with and storing the result of every calculation.

Function Table:

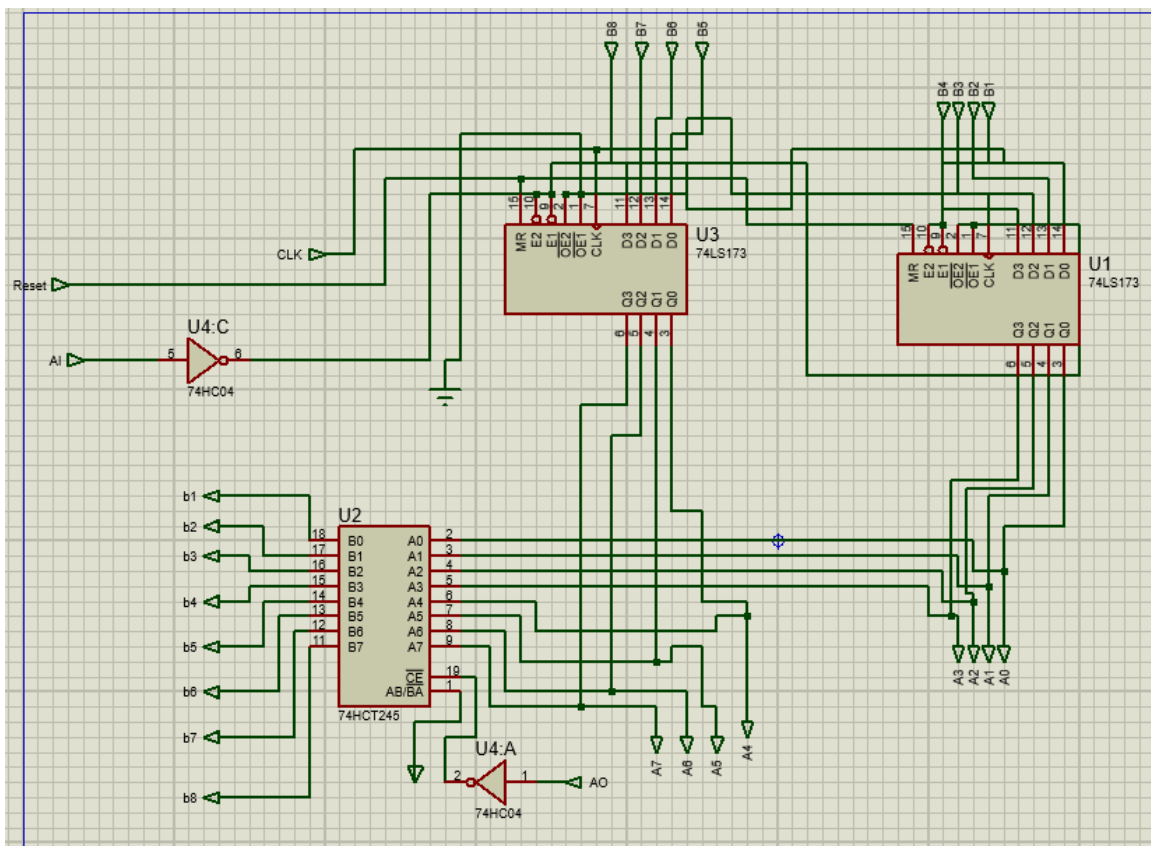
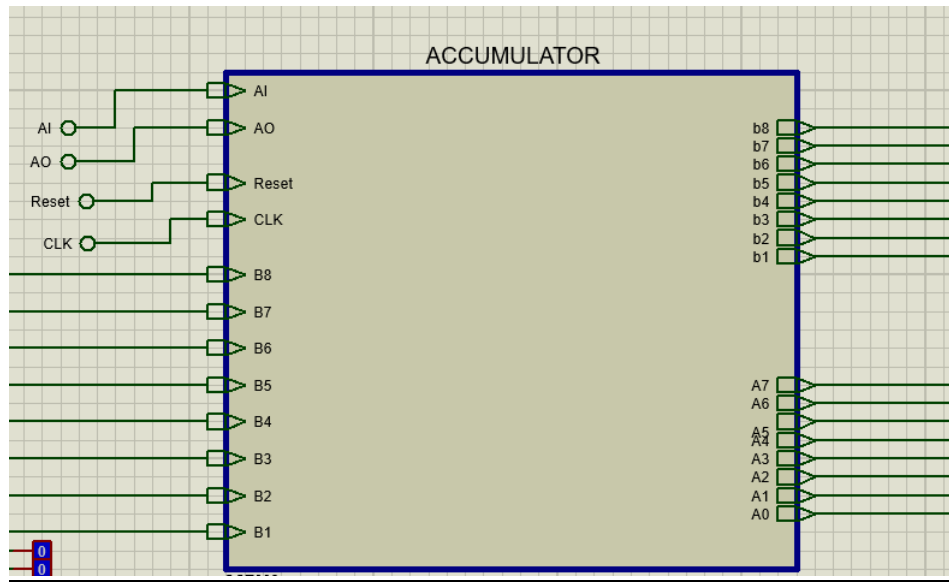
State	Reset	AI (In)	AO (Out)	Clock	Action
Reset	1	X	X	X	Clear data to 0000 0000
Hold	0	0	0	Trigger	No change; output stays off Bus

State	Reset	AI (In)	AO (Out)	Clock	Action
Load	0	1	0	Trigger	Save data from Bus
Output	0	0	1	Trigger	Send stored data to Bus

Accumulator Operational Algorithm

1. **Reset State:** When the **Reset** signal is high (1), the Accumulator ignores all other inputs and clears its internal data to 0000 0000.
2. **Hold State:** When **Reset**, **AI**, and **AO** are all low (0), the register maintains its current value without change. It remains isolated, and its output stays off the system bus.
3. **Load State (Bus to Accumulator):** To capture new data, the **AI (In)** signal must be set to 1 while **AO** and **Reset** are 0.
 - Upon the **Clock Trigger**, the 8-bit data currently present on the system bus is saved into the Accumulator's memory.
4. **Output State (Accumulator to Bus):** To send data back to the system, the **AO (Out)** signal must be set to 1 while **AI** and **Reset** are 0.
 - Upon the **Clock Trigger**, the stored 8-bit value is driven onto the system bus for use by other modules.
5. **Continuous Internal Output:** While not explicitly in the table, the Accumulator's stored value is constantly fed into the **Adder/Subtractor** module as one of its primary operands, as seen in your full SAP-1 schematic

Circuit Diagram:



B Register

The B Register is a temporary helper register used mainly for ALU operations.

- When the CPU performs operations like addition or subtraction, the B Register holds the second value needed for the ALU.
- Unlike the AC, the B Register usually does not connect to the bus, so it cannot directly send or receive data from memory or I/O devices.
- Its main role is to support the AC during computation, acting as a temporary storage place for one of the operands.

We can consider the B Register as the assistant of the AC because it holds the number that will be used in a calculation but does not store results permanently or communicate outside the CPU.

Function Table:

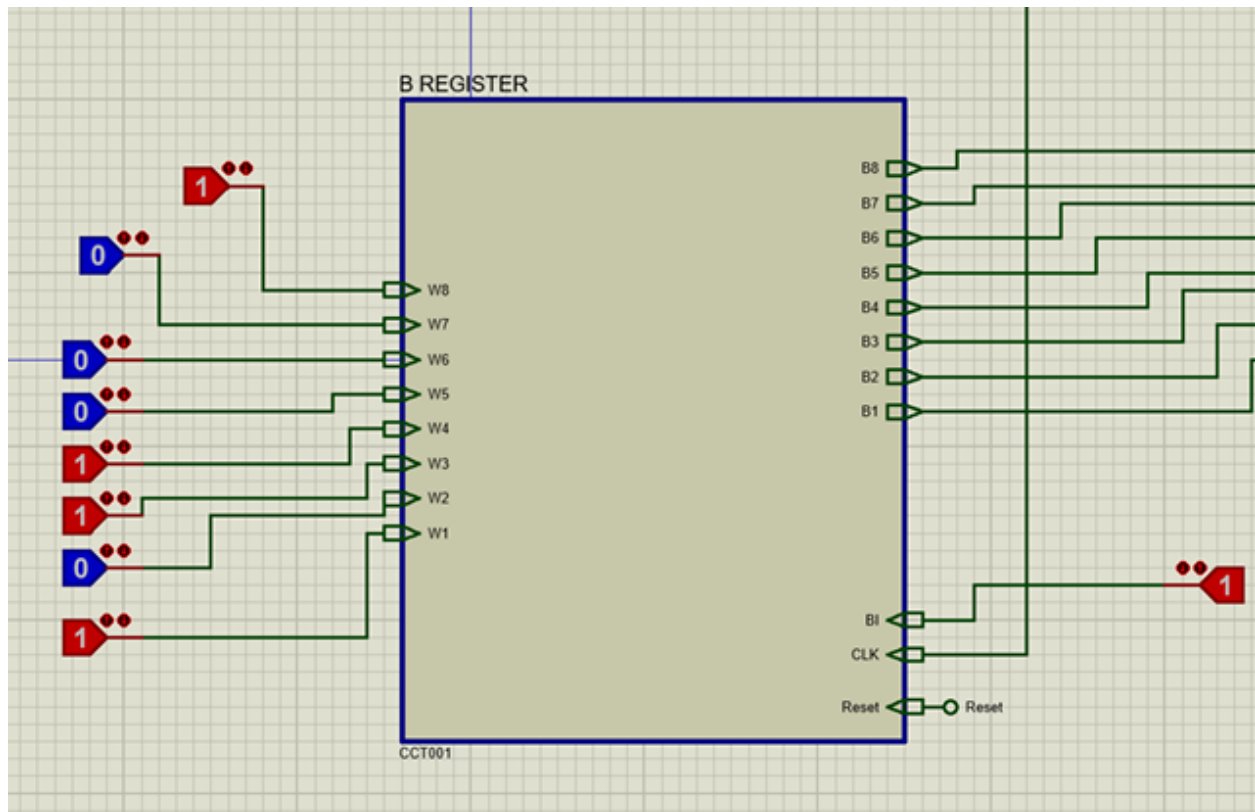
Operation Mode	Reset	B-In(BI)	Clock	Action
System Reset	1	X	X	All 8 bits become 0000 0000 immediately.
Hold Data	0	0	Trigger	No change. Keeps the old data.
Load Data	0	1	Trigger	Reads 8-bit data from the Bus and saves it.

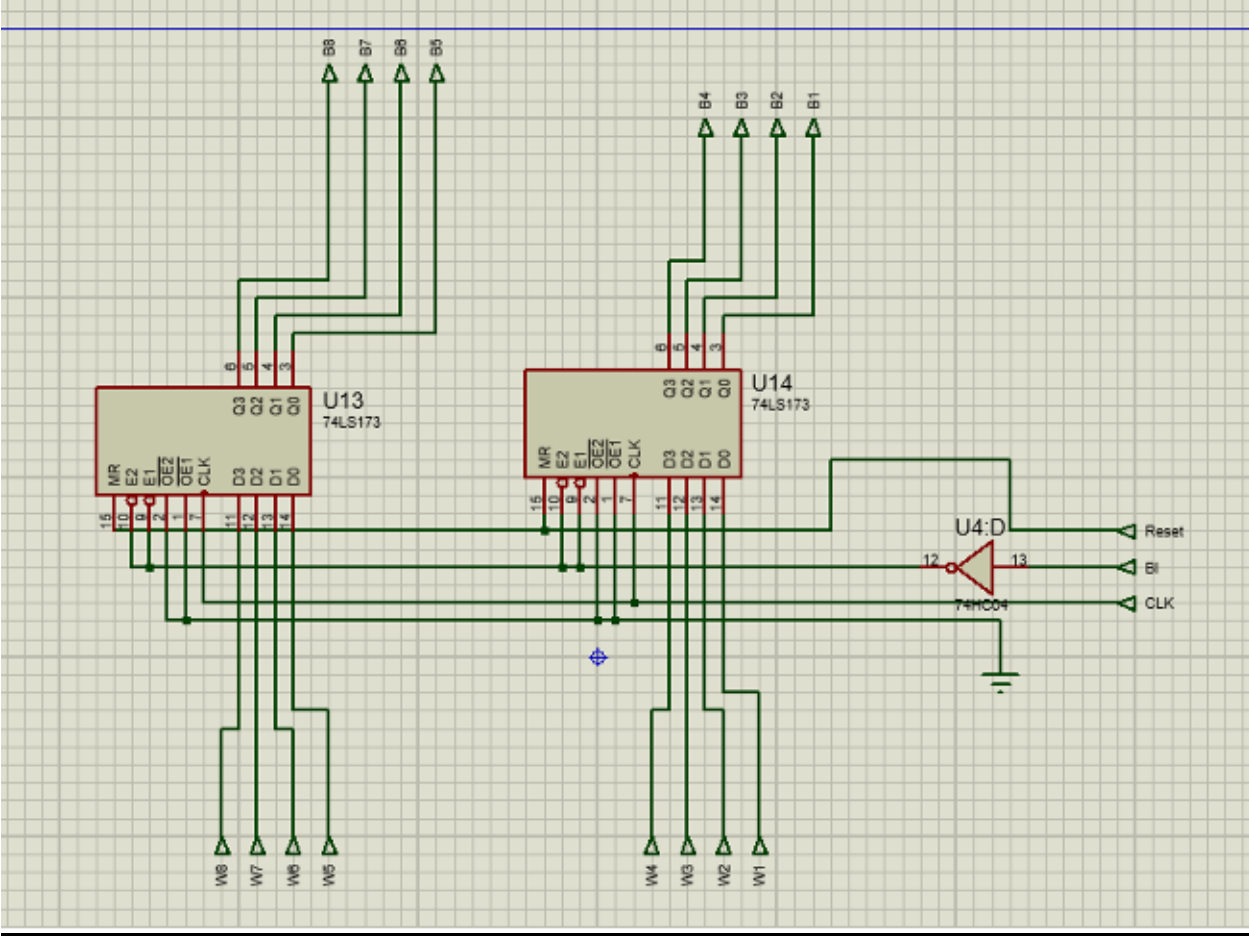
Algorithm of the B-Register:

1. Step 1 (Start): The register waits for a signal while holding its current 8-bit value.
2. Step 2 (Check Reset): If the Reset signal is High (1), the register clears all its internal memory to 0000 0000 instantly. It does not wait for the clock.
3. Step 3 (Check Control Signal): If Reset is 0, the register looks at the BI (B-In) signal.

4. Step 4 (Data Capture): When $BI = 1$ and a Clock Trigger occurs, the register "grabs" the 8-bit data currently present on the Data Bus.
5. Step 5 (Data Output): The stored 8-bit data is always sent to the ALU (Input B) for calculation. It never goes back to the Bus.
6. Step 6 (Loop): The register continues to hold this data until a new BI signal and Clock Trigger arrive, or the Reset is pressed.

Circuit Diagram:





RAM(Random Access Memory)

The RAM also known as Random Access Memory, is generally the primary volatile memory. But in case of our SAP1 8bit computer it is used to store both program instruction and data.

- This RAM has 16 addresses, each storing 8-bit data.
- The data can be stored or accessed by entering the suitable address.

BLOCK DIAGRAM:

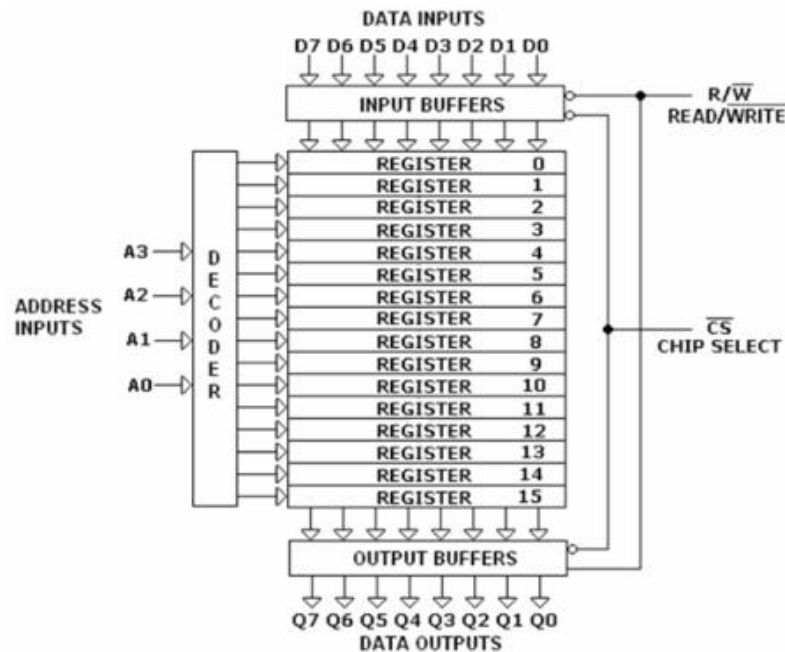


Fig. Blockdiagram of 74LS189

Introduction to 64bit RAM IC (74LS189):

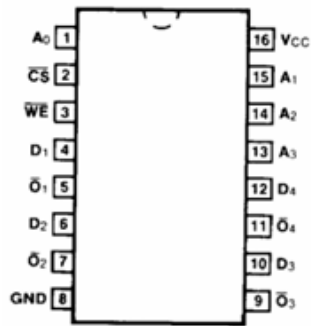


Fig. Pin diagram of 74LS189

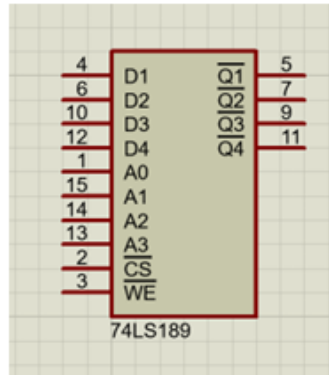


Fig. Schematic of 74LS189 in Proteus

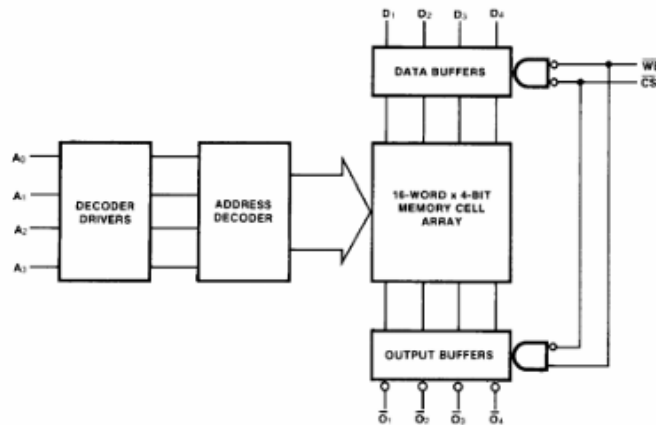


Fig. Logic diagram of 74LS189

The IC used in the design of the RAM is 74LS189. It is a 64bit static RAM IC that has in total 16 pins which are used as

- 4 address pin (A0, A1, A2, A3),
- 4bit data input (D1, D2, D3, D4)
- 4 inverted output of the stored data (Q1', Q2', Q3', Q4')

Additionally there are WE'(Write Enable) and CS'(Chip Select), where both of the are active low, and at last Ground and Power pin.

Function Table:

Input		Operation	Condition of Output
CS'	WE'		
0	0	Write	High Impedance
0	1	Read	Complement of stored data
1	X	Inhibit	High Impedance

Circuit Diagram:

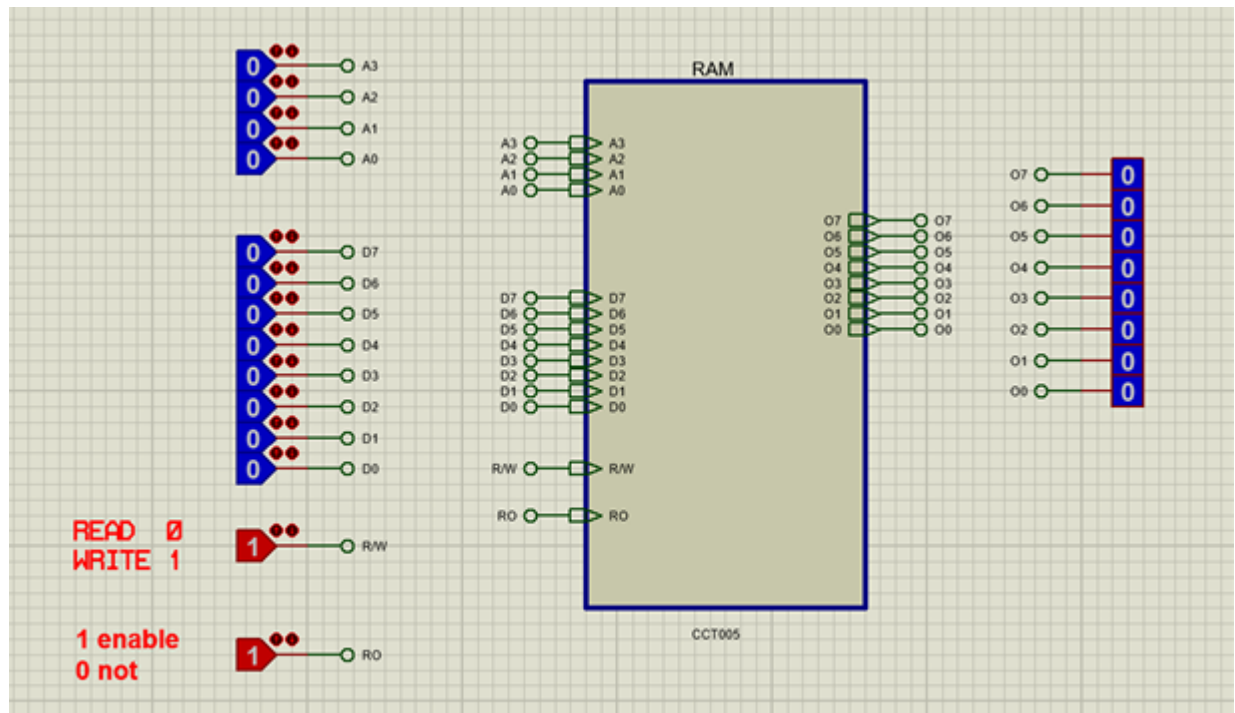


Fig. Subcircuit diagram of RAM

The pin in this circuit is similar to the one explained in the introduction section of the IC. There are two additional pins; R/W and RO.

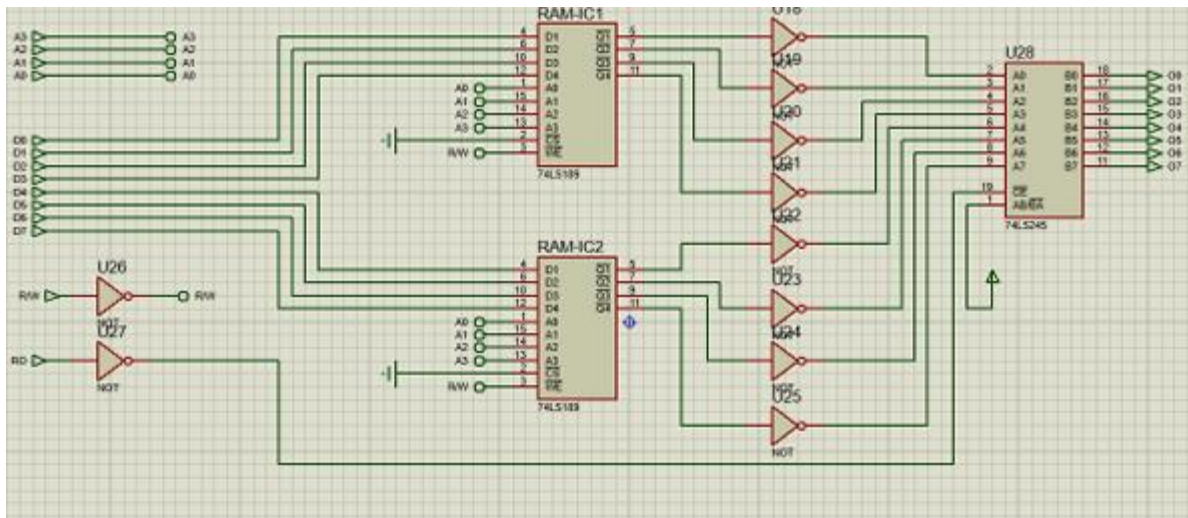


Fig: Childsheet circuit diagram of RAM

As we need to build a 16X8 bit RAM, we used two 64bit 74LS189 RAM ICs. The address pin in both of the ICs where shorted with the same name and input is given from one address. As we have 8 data bit each IC is responsible for 4bit of data. Again the each outputs are passed through a Inverter to get the actual stored data bit. At last the 8bit data from the IC is passed through a Buffer IC which is controlled by RO input.

Algorithm of the RAM:

The RAM designed in this project follows some instruction that is used to operate the RAM. They are given below,

1. R/W - selects if the IC is in read mode or write mode. 0- IC is in Read mode.
1- IC is in Write mode.
2. When R/W=0, then given 8bit data will be stored at the entered Address
When R/W=1, then stored data will be passed to output pin.
3. RO - RAM Out, indicates if the data in the output pin of the RAM would be passed to the bus or not.
0- Data doesn't get passed to bus.
1- Data get passed to bus.

Outcomes:

- Program Storage: RAM stores instructions and data required for program execution.
- Direct CPU Access: The Program Counter directly addresses RAM locations. Enables fast instruction fetch and data access.
- Simple and Reliable Operation : Small memory size keeps the design easy to understand and implement.

Limitations:

- Very Limited Memory Size: Only 16×8 of memory that can restricts program length. Even it is suitable for this kind of basic and simple computer architecture like SAP1, there should be a huge amount of change needed to be made so it is suitable for more suitable and complex computer architecture.
- No Separate Program and Data Memory: Same RAM stores both instructions and data. Can cause overwriting issues in larger programs.
- No Memory Protection: Risk of accidental data corruption.
- No Addressing Modes: Only direct addressing is supported. Limits flexibility in data access
- No Expand-ability: Memory size cannot be increased without major hardware changes.

Arithmetic and Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) serves as the primary calculation engine of the system, designed to perform fundamental arithmetic operations on 8-bit data. Its core functionality revolves around an 8-bit Adder-Subtractor circuit that utilizes 2's complement logic to handle both addition and subtraction seamlessly. By using XOR gates as programmable inverters, the ALU can either pass data from the B-Register directly for addition or flip the bits to create a 1's complement for subtraction. When the SU (Subtract) signal is active, the circuit adds a "1" to the least significant bit (LSB) via the carry-in (C0) of the first adder, completing the 2's complement process. The final 8-bit result is then interfaced with the system

bus through a 74HCT245 tri-state buffer, ensuring that the result is only driven onto the bus when the EO (Enable Out) signal is triggered by the controller.

A. Addition Strategy

The ALU uses binary full-adder logic. When the SU (Subtract) control signal is LOW (0):

- The XOR gates act as "buffers," passing the B-Register data to the adders without change..
- The carry-in (C0) of the lowest adder is 0.
- The system performs: $\text{Result} = A + B$.

B. Subtraction Strategy (2's Complement)

When the SU signal is HIGH (1), the ALU performs subtraction (by transforming it into $A + (-B)$).

1. Bitwise Inversion: XOR gates flip every bit of the B-Register (creating the 1's complement).
2. Adding the LSB: By feeding the SU signal into the carry-in (C0) of the first adder, we add 1 to the inverted B-value.
3. The system performs: $\text{Result} = A + (\bar{B} + 1) = A - B$.

Role of the XOR Gates (Programmable Inversion):

The XOR gate is the "Decision Maker" of the ALU. Its primary role is to control the B-Register's data based on the SU (Subtract) signal. In this ALU architecture, the XOR gate acts as a "Programmable Inverter." It is the critical component that allows the same adder hardware to perform both addition and subtraction •

Conditional Inversion (The Add/Subtract Switch):

By connecting one input of the XOR gate to the SU (Subtract) control signal, the gate determines how the ALU processes the B-Register data:

- Addition (SU = 0): When the SU signal is LOW, the XOR gate acts as a buffer. It passes the B-Register data to the adder without any modification.
- Subtraction (SU = 1): When the SU signal is HIGH, the XOR gate inverts every bit of the B-Register (converting 0s to 1s and 1s to 0s). This resulting value is known as the 1's Complement.

Implementing 2's Complement Logic:

To perform binary subtraction, we must use the 2's Complement method, defined as (1's Complement + 1). The XOR gate facilitates this in two ways:

1. It creates the 1's Complement by inverting the B-Register bits when SU=1.
 2. The same SU signal is fed into the Carry-In (C0) of the first Adder IC. This effectively adds the required "+1" to the inverted bits.
- Mathematical Result: $A + (\bar{B} + 1) = A - B$

Space and Cost Efficiency:

Without the use of XOR gates, the circuit would require separate Inverter ICs to flip the bits and Multiplexers to choose between the original and inverted data. The XOR gate performs both functions, signal passing and signal inversion simultaneously, making the circuit significantly more compact and power-efficient.

XOR Function Table:

The diagram shows a standard XOR gate symbol. It consists of two input lines on the left that curve inwards to meet at a point, and a single output line on the right. The symbol is a D-shaped outline with a curved front and a pointed tip.

Gate	Notation	Truth table																		
XOR	$A \oplus B$	<table><tr><th colspan="2">INPUT</th><th>OUTPUT</th></tr><tr><th>A</th><th>B</th><th>A XOR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	INPUT		OUTPUT	A	B	A XOR B	0	0	0	0	1	1	1	0	1	1	1	0
INPUT		OUTPUT																		
A	B	A XOR B																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	0																		

Input A (Control - SU)	Input B (Data - B Register)	Output (Y)
0	0	1
0	1	1
1	0	1
1	1	0

ALU Function Table:

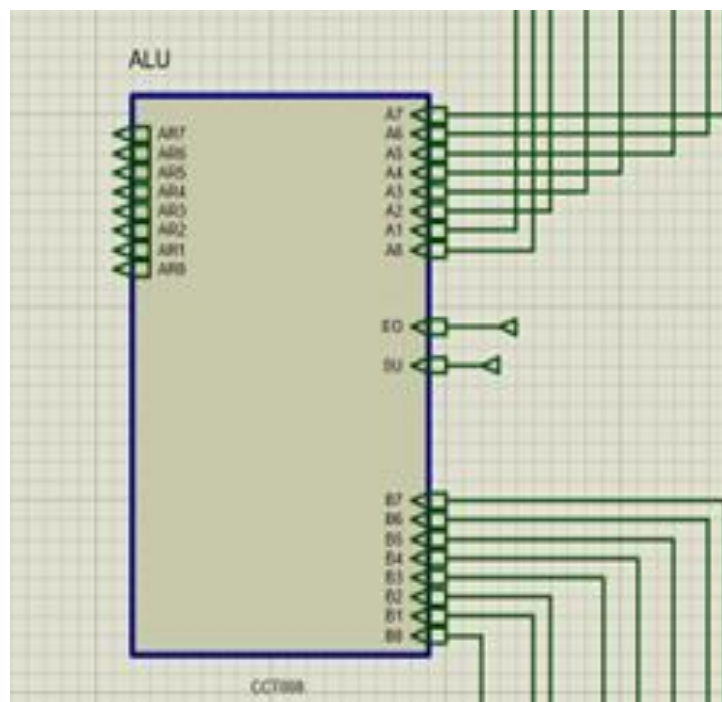
EO(Enable)	SU(Subtract)	ALU State	Result to Bus
------------	--------------	-----------	---------------

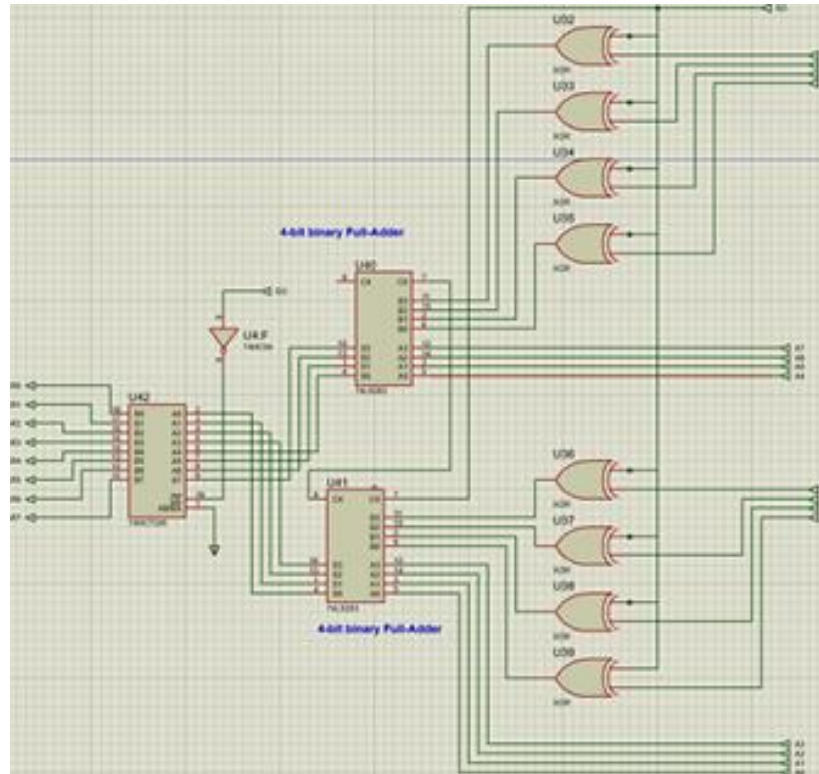
0	X	Isolated	High Impedance(Bus Disconneccted)
1	0	ADD	A+B
1	1	SUB	A-B

Algorithm Flow:

- A. Fetch: Load Register A and Register B from the system bus.
- B. Process: The Adders continuously calculate the sum/difference based on the SU signal.
- C. Output Enable: When the EO signal is triggered, the 74HCT245 activates, pushing the 8-bit result onto the bus pins (AR0-AR7)

Circuit Diagram:





Instruction Register

The Instruction Register (IR) is a critical 8-bit storage unit that facilitates the "Fetch" and "Execute" phases of the computer's operation. It is constructed using two 74LS173 4-bit D-type registers, which capture data from the system bus when the II (Instruction In) signal is activated during the fetch cycle. Once loaded, the IR splits the 8-bit word into two distinct functional parts: the upper 4 bits (Opcode) and the lower 4 bits (Operand). The Opcode bits are sent directly to the Control Unit to determine the operation to be performed, while the Operand bits (typically a memory address) can be outputted back onto the bus via the IO (Instruction Out) signal. This dual-output design allows the IR to act as a bridge between the software instructions stored in memory and the hardware execution units of the CPU.

The Instruction Register (IR) in the SAP-1 is an 8-bit, parallel-in/parallel-out register. While it functions similarly to the other registers in the system, its unique output configuration sets it apart.

Here is a breakdown of how it works:

- Loading Data: The Instruction In signal acts as the control switch. When activated, it allows the 8-bit instruction to flow from the bus into the register. (This is similar to the Shift/Load logic used in the MAR).
- The Split Output: Once the instruction is stored, it is divided into two 4-bit sections (nibbles):
 - The First Nibble (Opcode): This goes straight to the Controller-Sequencer to tell the computer what operation to perform.
 - The Second Nibble (Address): This is sent through a tri-state buffer. When the IO (Instruction Out) signal is triggered, this address is released back onto the bus, where it is typically picked up by the MAR for memory access.

IR Operation Algorithm:

The IR operates based on the control signals provided in your diagram (II, IO, CLK, and Reset):

1. Resting State: The IR ignores the bus as long as the II (Instruction In) signal is inactive.

2. The Fetch Step:

- The Control Unit sets II = 0 (Active Low for 74LS173).
- On the next Rising Edge of the CLK, the 8-bit data currently on the bus is latched into U44 and U45.

3. Instruction Output:

- The Opcode bits are permanently outputted from pins 3-6 (Q0-Q3) to the Controller.
- o If the instruction requires an address (Operand), the IO (Instruction Out) signal is activated to push those bits onto the bus.

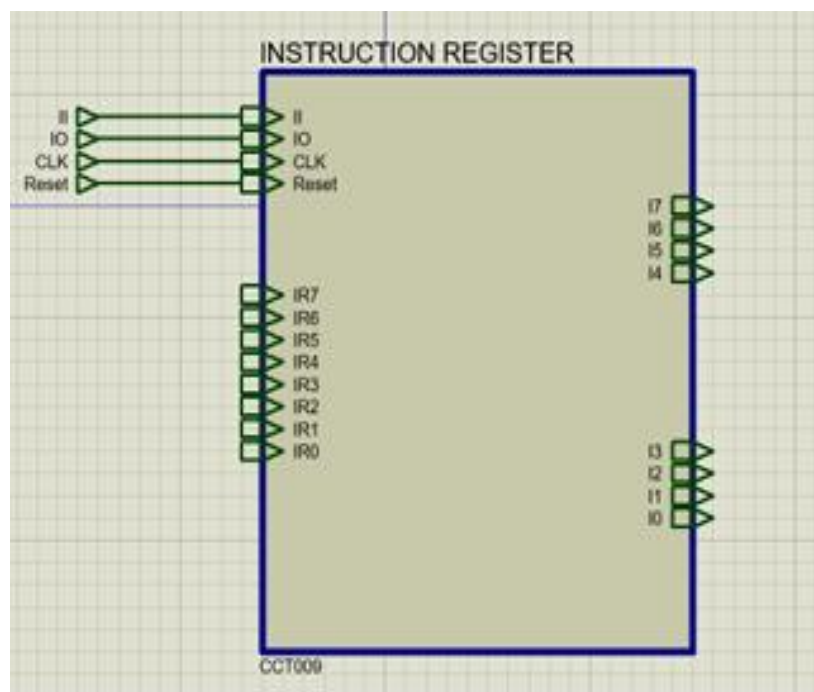
4. Reset: Activating the Reset line clears the current instruction to 0000 0000

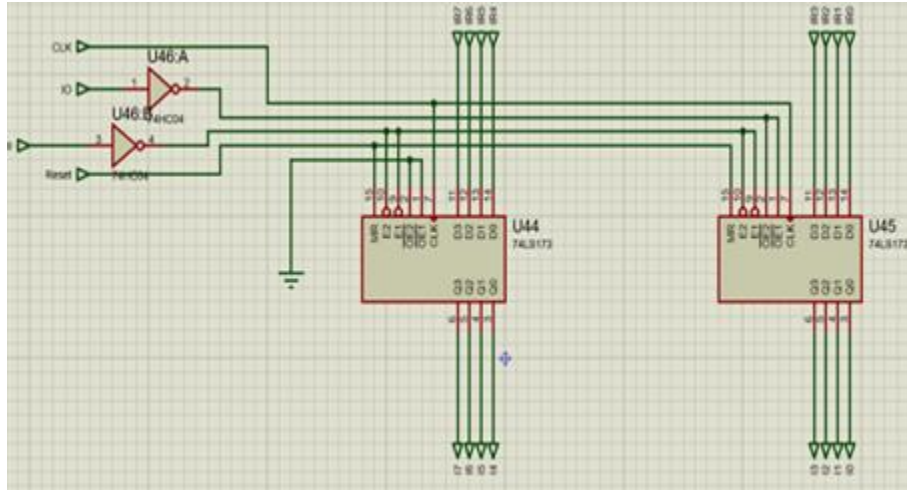
Function Table:

Reset	II(In)	IO(OUT)	CLK	State/Operation
-------	--------	---------	-----	-----------------

1	X	X	X	Reset: All internal bits become 0.
0	1	X	Trigger	Latched: IR holds previous data; ignores bus.
0	0	X	Trigger	Load: IR captures 8-bit data from the bus.
0	X	1	X	Output Disabled: Bus lines IR0-IR3 are Hi-Z.
0	X	0	X	Output Enabled: Operand bits move to bus.

Circuit Diagram:





CONTROL SEQUENCER

The control sequencer acts as the main component that coordinates the movement of data across the common bus translating machine instruction into timed electrical signal. It decides which component sends data to the bus and which on will receive that data for each of the cycle. According to the need it enables or disables different control pins of the several needed components depending. The component depends on the clock pulse from Clock Pulse Generator and takes 4bit input from the higher nibble of the Instruction Register which will direct it to some specific command to be executed.

In this computer architecture, the control sequencer defines the microinstructions and provides control signal output for each machine cycle. The block diagram for it is given below,

Block Diagram:

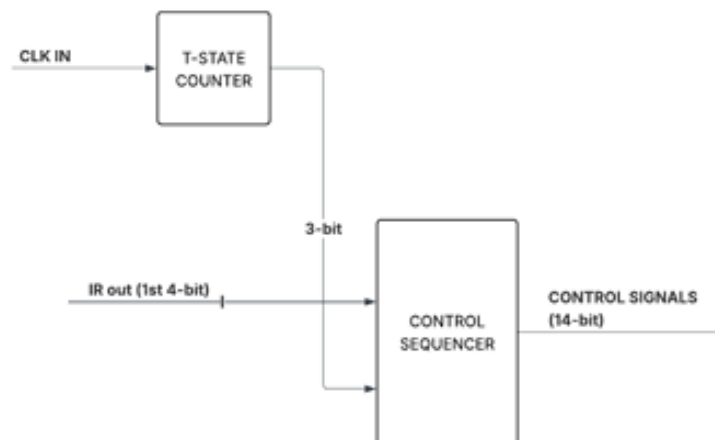


Fig: block diagram of control sequencer

T-States, Fetch, and Execution Cycles:

T-states (Timing states) represent individual clock cycles within a machine cycle. In the SAP-1 architecture, a single instruction is broken down into a sequence of these micro-steps to coordinate the movement of data between registers.

- The T-State Counter: A 3-bit counter (like the 74LS161) cycles through binary values 000 to 101 (T0 to T5). Each count triggers a specific "Control Word" from the EPROM.
- The Fetch Cycle: This occurs during T0 and T1. It is universal for every instruction. The address is moved from the Program Counter to the Memory Address Register, the instruction is read from RAM, and it is loaded into the Instruction Register.
- The Execution Cycle: This occurs during T2, T3 and T4. The actions taken here depend entirely on the 4-bit opcode currently in the Instruction Register (e.g., LDA, ADD, SUB). During these states, the control unit toggles specific pins to perform the math or data transfers required by that specific command.

THE PROM:

[NOTE: For hardware implementation we would use the 28C16 EEPROM, as it is much easier to erase and reprogram. But this component is unavailable in proteus. So we instead used a 27C64 EPROM for software implementation.]

The 27C64 is an 8KB (64K-bit) UV-erasable Programmable Read-Only Memory. We are using the EPROM as a Combinational Logic Replacement. Instead of using dozens of logic gates to decode instructions, we use the EPROM's address pins as inputs and its data pins as outputs.

- Inputs (Address Pins): we connected the 4 bits from the Instruction Register and the 3 bits from the T-state counter to the address lines (A0–A6).
- Outputs (Data Pins): For every unique combination of "Instruction + Time," the EPROM outputs a 16-bit (using two chips in parallel) Control Word.

Control Pin Definitions:

- HLT: Stops the clock
- MI: MAR in

- RI: RAM in
- RO: RAM out
- IO: IR out
- II: IR in
- AI: Accumulator in
- AO: Accumulator out
- EO: sum out
- SU: Subtract
- BI: B REG in
- OI: Output REG in
- CE: Count enable
- CO: Counter out

The Binary File Structure:

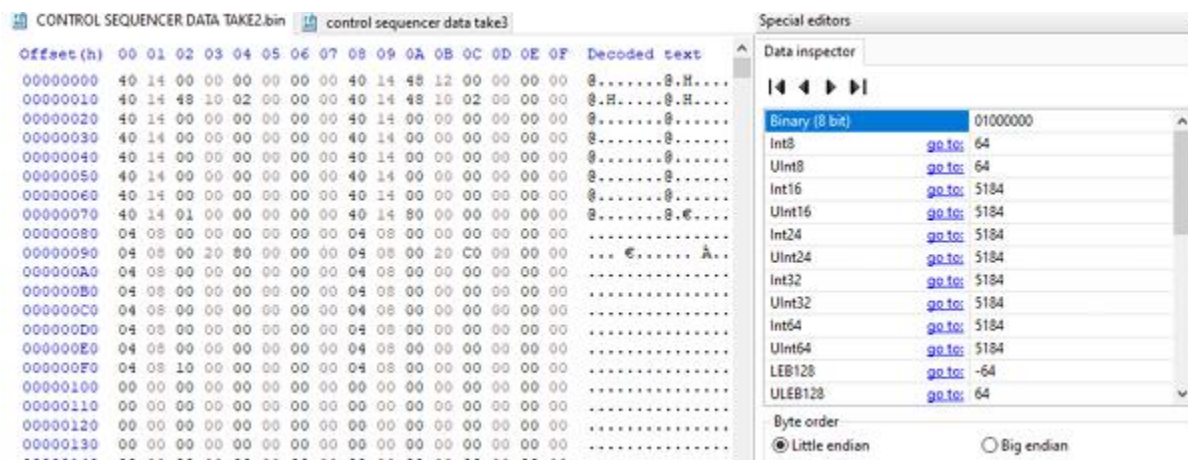


Fig: bin file written in HxD

A bin file is a raw data image. In the context of your 27C64 EPROM, it represents the exact state of every memory cell within the chip.

- Address Mapping: Each row in your HxD editor corresponds to a physical address in the EPROM. Since you are using the 4-bit Instruction Register and 3-bit T-state counter as inputs, these bits form the address.
- Data Values: The hex values you typed into HxD are the Machine Control Words. Each byte (or pair of bytes) represents the 1 or 0 state of the control pins (HLT, MI, RI, etc.) for that specific instruction and time step.

The Coding Process in HxD:

Creating this file involves translating your handwritten truth table into hexadecimal notation.

1. Mapping Bits to Pins: We assign each control signal to a specific bit in a byte. For instance, if MI is Bit 7 and RI is Bit 6, and we want only MI active, we would calculate the binary value 10000000.
2. Padding: Because a 27C64 has 8KB of space and your SAP-1 logic only uses a small fraction of that (the first 128 addresses if using 7 address bits), the rest of the bin file is usually filled with 00 to fill the remaining capacity of the chip.
3. Firmware Generation: Once saved, this .bin file acts as the virtual "firmware" that Proteus loads into the EPROM component.

CONTROL ROM TRUTH TABLE:

Instruction and fetch cycle	Opcode	STEP	HLT	MI	RI	RO	IO	II	AI	AO		EO	SU	BI	OI	CE	CO
fetch	XXXX	000	0	1	0	0	0	0	0	0		0	0	0	0	0	1
		001	0	0	0	1	0	1	0	0		0	0	0	0	1	0
LDA		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	1	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0
ADD		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	0	0		0	0	1	0	0	0
		100	0	0	0	0	0	0	1	0		1	0	0	0	0	0
SUB		010	0	1	0	0	1	0	0	0		0	0	0	0	0	0
		011	0	0	0	1	0	0	0	0		0	0	1	0	0	0
		100	0	0	0	0	0	0	0	1	0		1	1	0	0	0
OUT		010	0	0	0	0	0	0	0	1		0	0	0	1	0	0
		011	0	0	0	0	0	0	0	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0
HLT		010	1	0	0	0	0	0	0	0		0	0	0	0	0	0
		011	0	0	0	0	0	0	0	0		0	0	0	0	0	0
		100	0	0	0	0	0	0	0	0		0	0	0	0	0	0

Circuit Diagram:

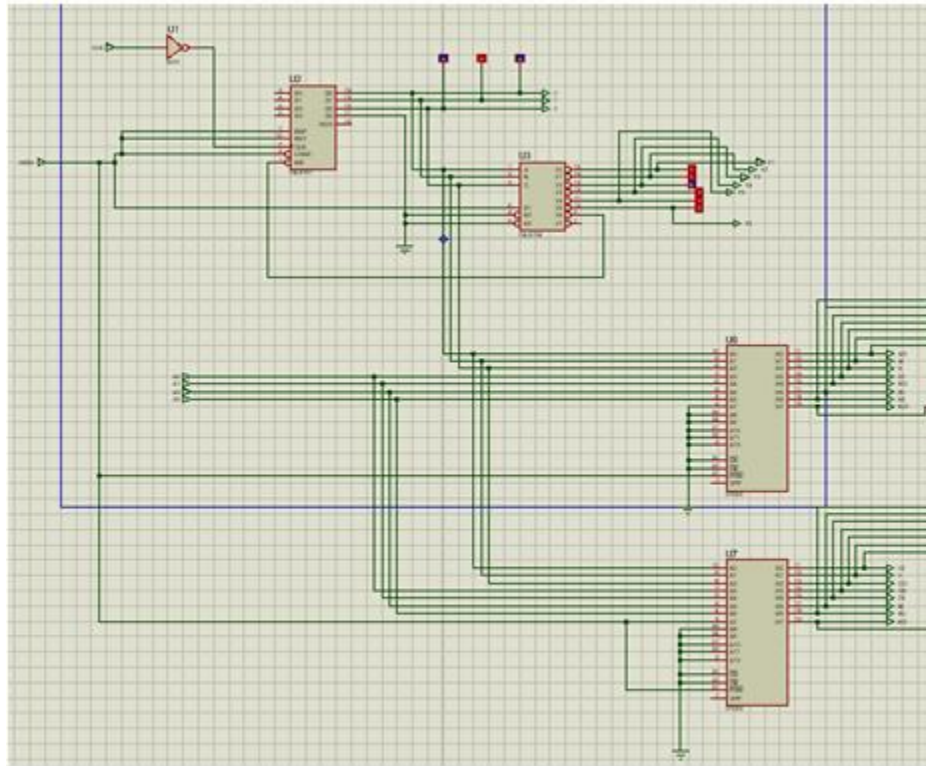


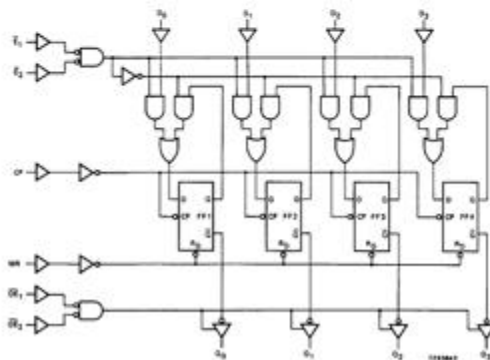
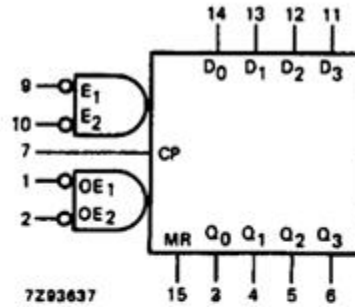
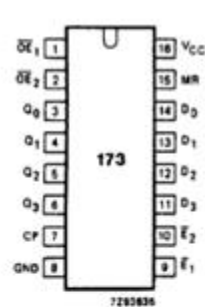
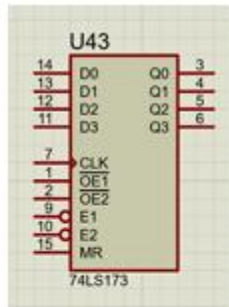
FIG: CONTROL SEQUENCER DIAGRAM IN PROTEUS

MAR & Address Selector

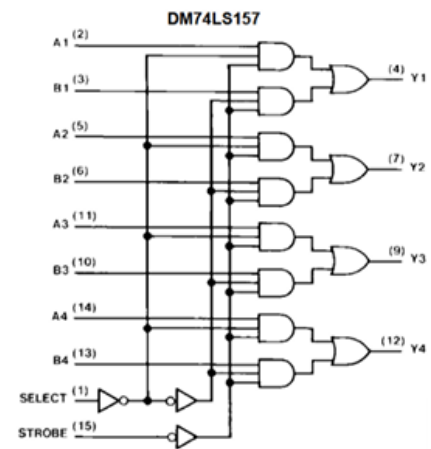
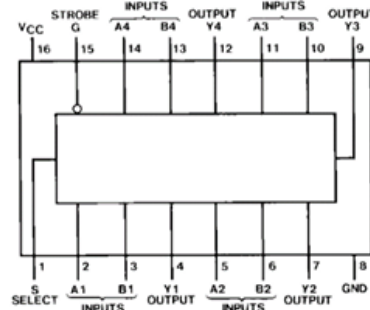
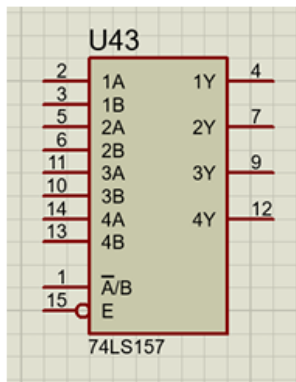
In SAP1 computer, the MAR and Address Selector work together to decide depending on the cycle of operation which source of address should be feed into the RAM. We can say that they are general components used for managing memory operation allowing the processor to access specific data location.

- MAR (Memory Address Register): A 4bit register that stores the memory address currently being accessed.
Function: During Fetch cycle, it gets address from Program Counter via bus and during Execution cycle it gets the address from Instruction Register.
- Address Selector: A quad 2-to-1 MUX that selects which address is passed to RAM. Mode of Operation:
 1. Program mode: Allows manual selection.
 2. Run mode: Selects the address passed from MAR.

Introduction to 4bit D-type Flip Flop IC or 74LS173:



Introduction to Quad 2-to-1 MUX IC or 74LS157:



Function Table:

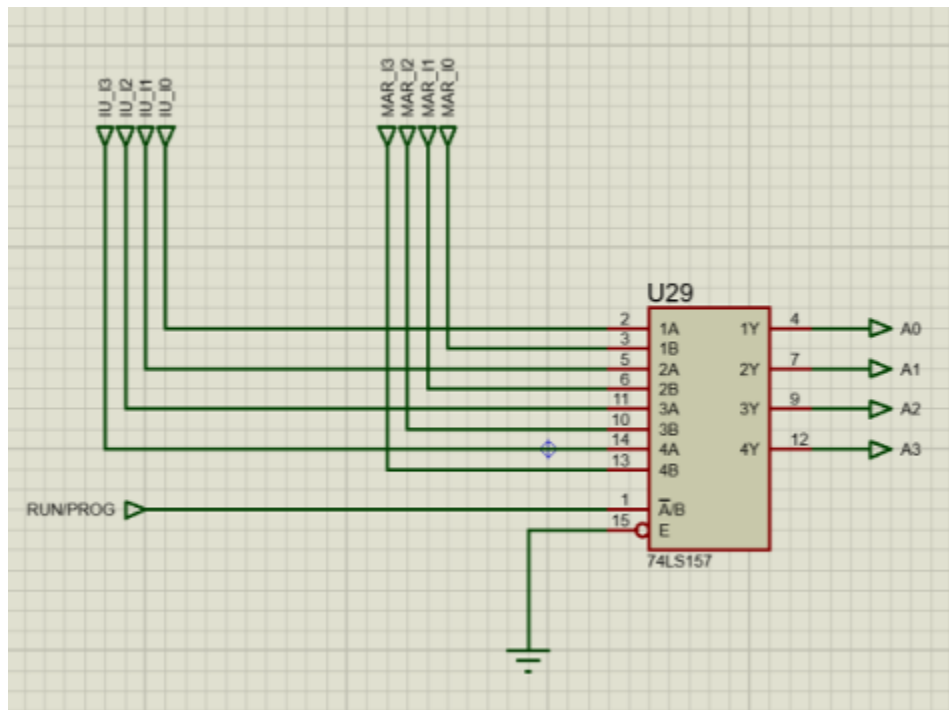
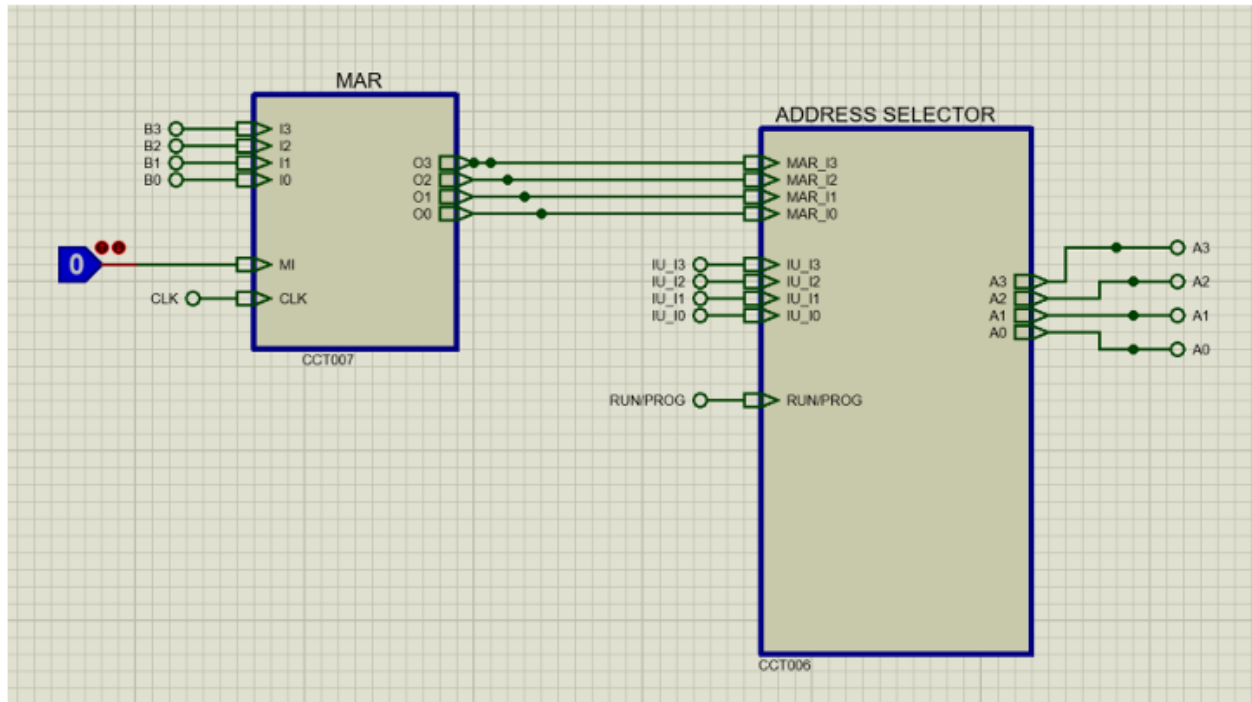
Fig. For 74LS173,

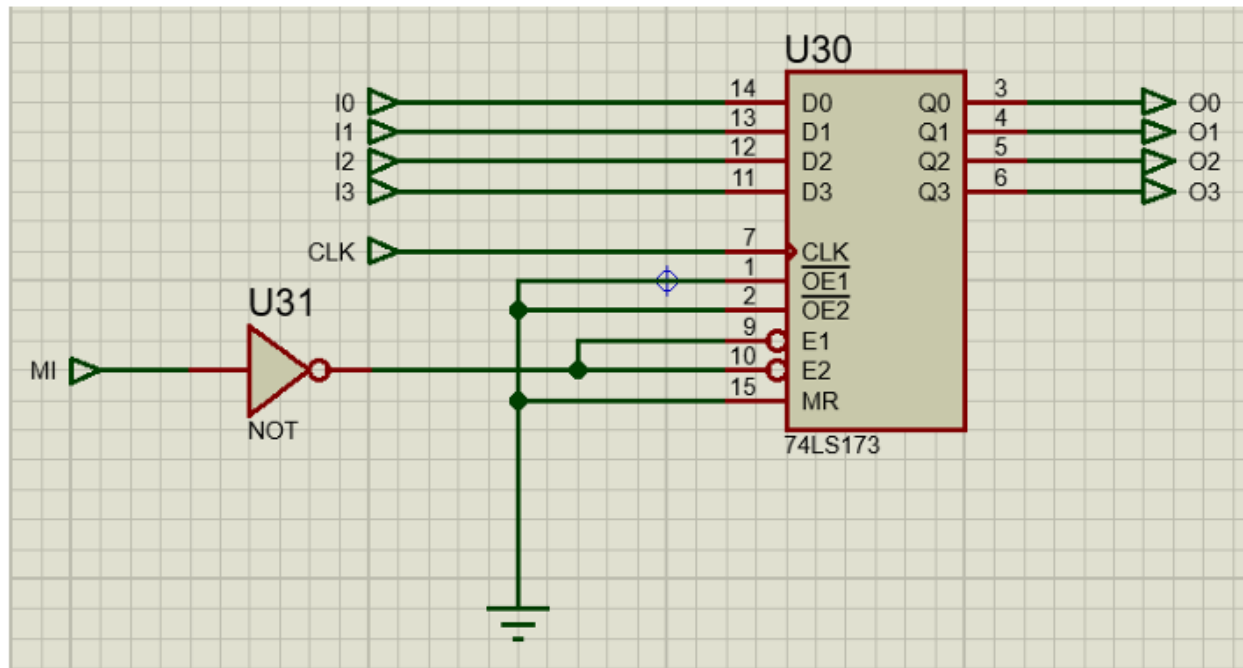
MR	CLK	E1'	E2'	OE1'	OE2'	Condition
1	X	X	X	X	X	No output
0	+ve edge	0	0	0	X	No output
				X	0	
		0	X	0	0	
		X	0			
		0	0	X	X	Data from bus gets stored
				0	0	Stored data shown at output

Fig. For 74LS157,

E	A'/B	Input		Output
		A	B	
1	X	X	X	0
0	0	Desired input		Shows A
	1			Shows B

Circuit Diagram:





Algorithm of MAR & Address Selector:

- When $MI=0$, no output is shown
- if $MI=1$ then data from bus is passed to Address Selector at each clock pulse.
- Address selector gets two 4bit address; one from MAR and other from Input Unit. • When $RUN/PROG=0$, it is in Program mode which means the address from input unit will be passed to RAM.
- If $RUN/PROG=1$, it is in RUN mode which means address from MAR will be passed to RAM.

Output Register

DESCRIPTION:

The Output Register is an 8-bit storage module designed to latch results from the system bus and present them for user observation. Utilizing two 74LS173 registers, it captures data on the Rising Edge of the clock only when the OI (Output In) control signal is activated. To ensure hardware compatibility, a 74HC04 inverter polarizes the high-level control signal into an active-low enable for the registers. Because the output enable pins are permanently grounded, the stored value is continuously displayed on the OO0-OO7 pins, providing a stable visual readout of the computer's final computational output.

- **Data Input (OI0-OI7):** These lines are connected to the 8-bit system bus.
- **Data Output (OO0-OO7):** These are the output lines that typically drive an LED bar or a 7-segment display driver. Unlike other registers, the output here is permanent (connected to ground via OE pins) so the user can always see the value.
- **Control Logic:** The register is controlled by the OI (Output In) signal. This signal passes through a 74HC04 hex inverter (U1:A) before reaching the active-low enable pins (E1 and E2) of the 74LS173 chips. This ensures that when the control unit sends a HIGH signal for OI, the registers become active to load data.
- **Clock and Reset:** The module is synchronized with the system CLK and can be cleared using the Reset line connected to the Master Reset (MR) pins.

Function Table:

Reset (MR)	OI (Output In)	CLK	Data Bus (OI0-OI7)	Logic State	Output (OO0-OO7)
High (1)	X	X	X	Reset Mode	0000 0000
Low (0)	Low (0)	Trigger	D	Hold Mode	No Change (Last Value)
Low (0)	High (1)	Trigger	D	Load Mode	Updated to D
Low (0)	X	Low/High	X	No Clock	No Change

Operation Algorithm

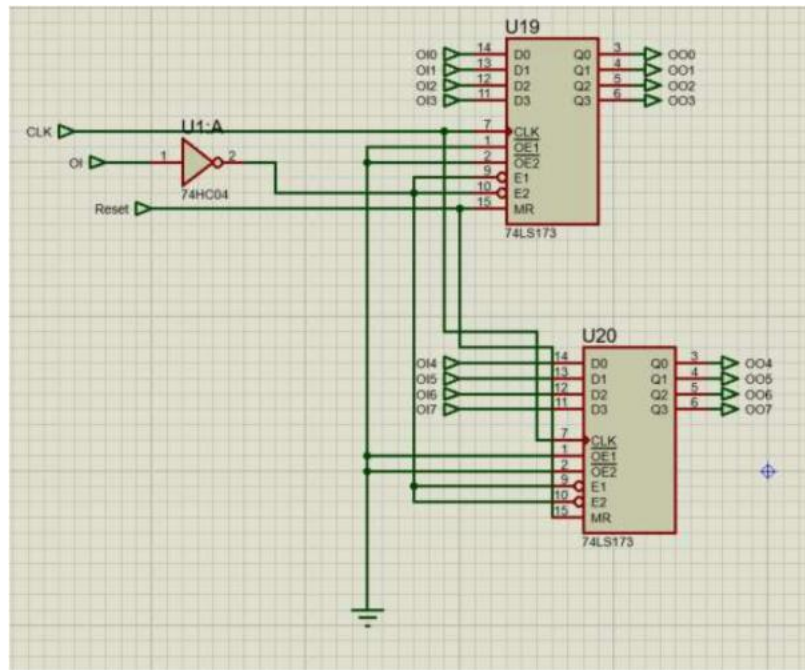
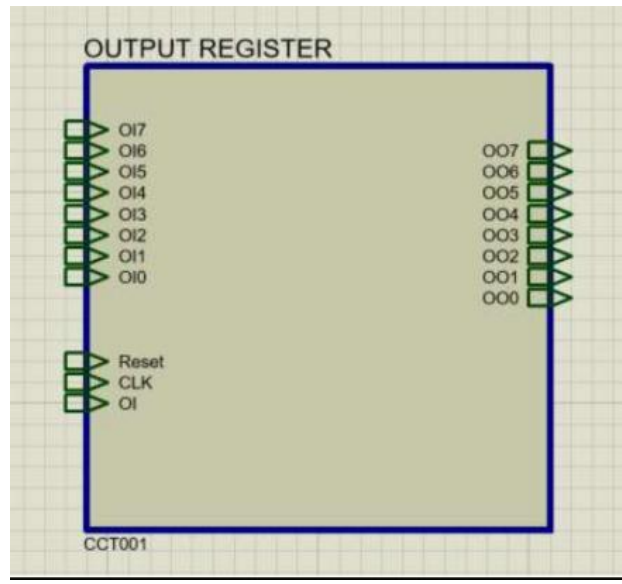
This algorithm describes the step-by-step logic the hardware follows to display a result:

- 1. Idle State:** The OI signal is kept LOW (Logic 0). The inverter (U1:A) outputs a HIGH to the enable pins, causing the registers to ignore any data changes on the system bus.
- 2. Enable Data Capture:** The Control Unit triggers the OI signal HIGH (Logic 1).
- 3. Inversion:** The 74HC04 inverter flips the signal to LOW (Logic 0), enabling the E1/E2 pins of U19 and U20.
- 4. Latch on Clock:** On the next Rising Edge of the system CLK, the 8-bit data currently on the bus (OI0-OI7) is latched into the internal flip-flops of the registers.

5. Permanent Display: Since the Output Enable pins are permanently tied to Ground, the latched 8-bit data is immediately and continuously visible on the output pins (**OO0-OO7**).

6. Reset (Optional): If the Reset line is pulled HIGH, the MR pins clear the registers, forcing the output to 0000 0000.

Circuit Diagram:



Limitations:

The primary limitation of this output register design is its volatility, meaning all displayed data is immediately lost if the system is reset or powered down. Additionally, because the Output Enable pins are permanently grounded to provide a continuous display, the module is strictly a "one-way" destination and cannot write data back onto the system bus for reuse in other calculations. The circuit also lacks Asynchronous Loading capabilities, meaning it is entirely dependent on the system clock to update its state, which can lead to display lag if the clock speed is extremely low. Finally, without an onboard Binary-to-Decimal decoder, the output is restricted to raw binary format (via LEDs), making it less intuitive for users who require decimal or hexadecimal readouts.

Integration with 8-bit bus

BUS IMPLEMENTATION:

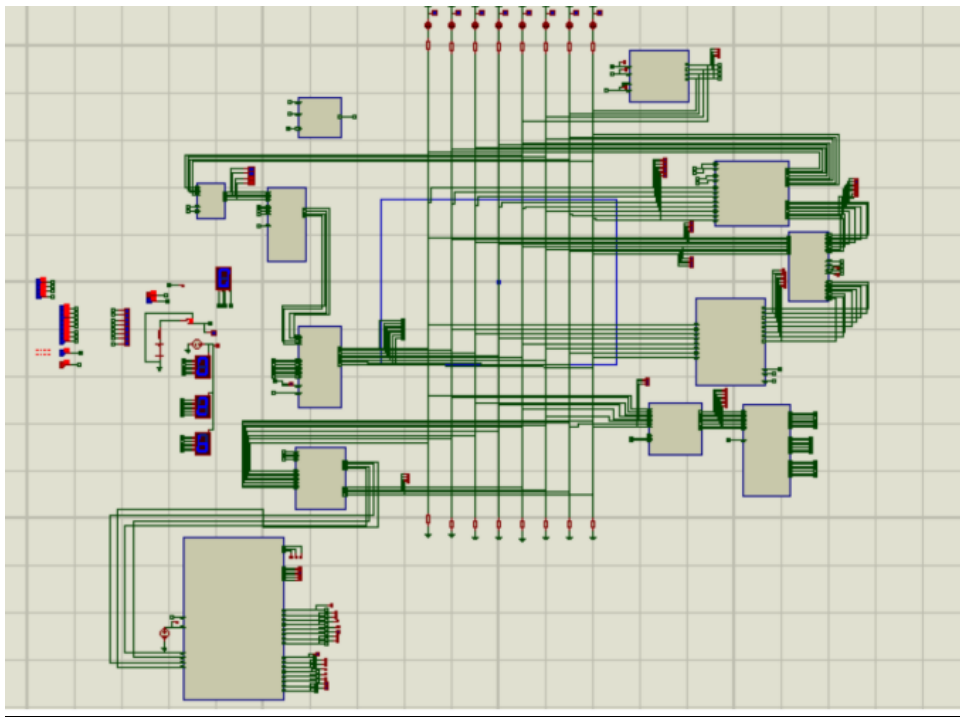


FIG: All components connected to the bus

In the SAP-1 architecture, the Bus is the central nervous system of the computer. It is a single 8-bit parallel communication path that connects all major functional blocks (Registers, ALU, Memory, etc.).

Since only one component can "talk" (place data) on the bus at any given time to avoid short circuits (bus contention), each part uses Tri-state Buffers to connect or disconnect from the lines.

Bus Definition:

The bus is a collection of 8 wires labeled D0 through D7. It operates on the principle of Time-Division Multiplexing, where the Control Unit ensures that only one "Output Enable" signal is active during any specific T-state of the machine cycle.

2. Individual Part Implementations

The Accumulator (Register A)

- **Input:** Connected to the bus via D-type flip-flops. It loads data from the bus when the LDA(Load A) signal is low.
- **Output:** Connected to the bus through tri-state buffers. It places its stored byte onto the bus only when EA (Enable A) is active.
- **Internal Link:** It also has a hardwired (non-bus) connection directly to the ALU inputs.

The B Register

- **Input:** Loads data from the bus when L_B is active.
- **Output:** Unlike Register A, the B Register typically does not have an output to the bus. It exists solely to feed the second operand to the ALU logic gates.

The ALU (Arithmetic Logic Unit)

- **Input:** Receives operands directly from Register A and Register B (bypassing the bus for speed).
- **Output:** The result (Sum, Difference, AND, or OR) is sent to the bus through the OR- merger gate array and tri-state buffers. This output is gated onto the bus only when the EU (Enable Unit) signal is triggered.

Program Counter (PC)

- **Output:** The 4-bit address stored in the PC is placed onto the lower nibble of the bus (D0- D3) when EP is active, allowing the MAR to fetch the next instruction address.

Memory Address Register (MAR)

- **Input:** It "snaps" the address from the bus when LM is active. It does not have a bus output; its output is hardwired to the RAM address pins.

RAM (Random Access Memory)

- **Output:** When the CE (Chip Enable) signal is active, the RAM retrieves the byte at the address specified by the MAR and places it onto the 8-bit bus for the Instruction Register or Accumulator to read.

Instruction Register (IR)

- **Input:** Loads the instruction from the bus when LI is active.
- **Output:** The lower 4 bits (operand/address) are placed back onto the bus when EI is active, while the upper 4 bits (opcode) go directly to the Controller/Sequencer.

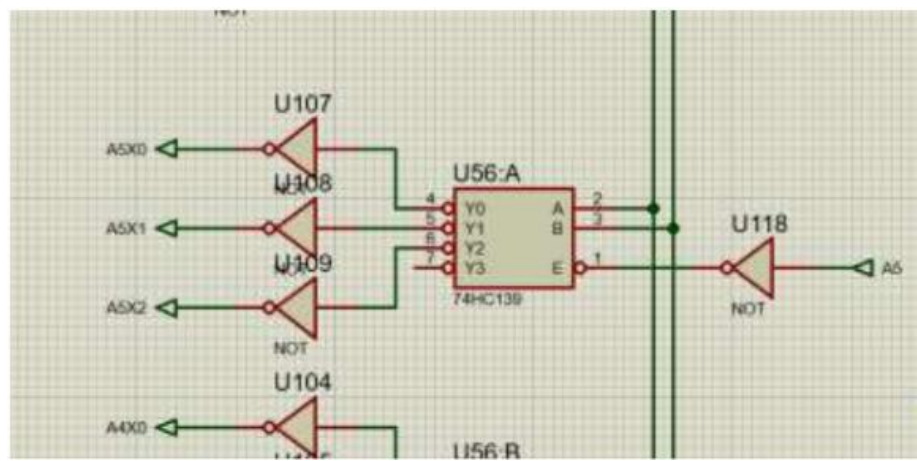
FEATURE

(Modification of ALU):

For our feature, we added 3 new instructions to SAP-1. Two of them were to add a bitwise AND and a bitwise OR operation. To do this, we needed to modify the ALU. We did this by adding 3 ICs. These are:-

1. 74HC139 (2-to 4-line demultiplexers)
2. 8 two input AND gates
3. 8 two input OR gates

DESCRIPTION:



This modification transitions the standard SAP-1 ALU from a purely arithmetic unit to a multifunctional Arithmetic Logic Unit (ALU) by incorporating bitwise logical operations.

1. Control Logic & Signal Routing

The system uses a Demultiplexer (74HC139) acting as a decoder to steer the data flow based on specific control pins (Address lines A and B).

- **Input Splitting:** The 8-bit inputs are routed in parallel to three distinct sub-units:
- **Arithmetic Unit:** Utilizing 74LS283 4-bit Binary Full-Adders for addition and subtraction.
- **Bitwise AND Unit:** A bank of AND gates (U131–U138) processing INA . INB.
- **Bitwise OR Unit:** A bank of OR gates (U139–U146) processing INA + INB

2. Operation Selection

The Demultiplexer serves as the operation selector. By toggling the address lines, the ALU activates only the desired computation path:

CONTROL PIN (ADDRESS)	ACTIVE OPERATION	COMPONENT
00	ARITHMETIC	74LS283
01	AND	U131-U138 AND Gates
10	OR	U139-U146 OR Gates

3. Intermediate Output Stage

Before the final result is sent to the Accumulator bus, the signals pass through an OR-gate array (U77, U85–U91). These gates act as a hardware "merger," collecting the outputs from the AND, OR, and Adder blocks. Since the Demultiplexer ensures only one operation is active (logical high) at a time while others remain at high impedance or logical low, the merger outputs the result of the currently selected function without bus contention.

4. Bitwise Implementation

- **XOR Modulation:** The XOR gates (U36–U39) remain integrated with the 74LS283 units to facilitate Two's Complement subtraction when the SUB control line is active.
- **Parallel Processing:** All logic operations are performed asynchronously; the Demultiplexer simply determines which result is gated through to the output.

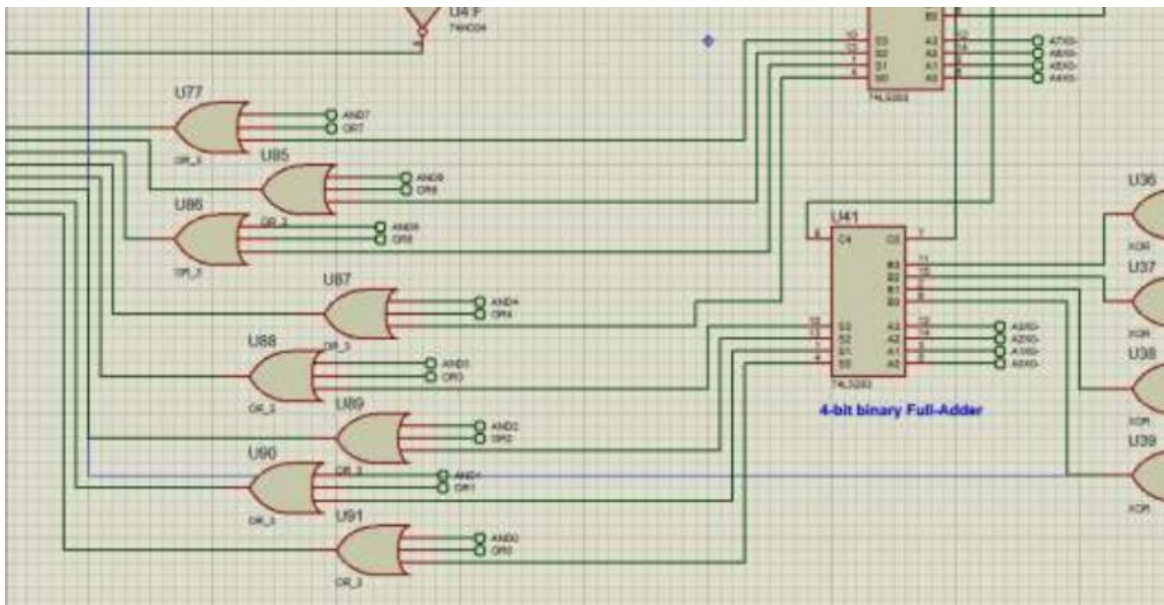


FIG: adding the 3 operations using OR gate

MICROCODE:

INSTRUCTION	OPCODE8	T-STATE	CONTROL PINS ON
AND	0100	T2	IO MI
		T3	RO BI
		T4	EO AI AN
OR	0101	T2	IO MI
		T3	RO BI
		T4	EO AI OR
MVI	0110	T2	IO AI

Jump Instruction

To facilitate a non-sequential program flow (JUMP), the original asynchronous Program Counter (PC) was replaced with a **synchronous 4-bit binary counter** modeled after the **74LS163A**. This architectural shift allows the PC to transition from a simple incrementer to a loadable register capable of capturing addresses directly from the bus.

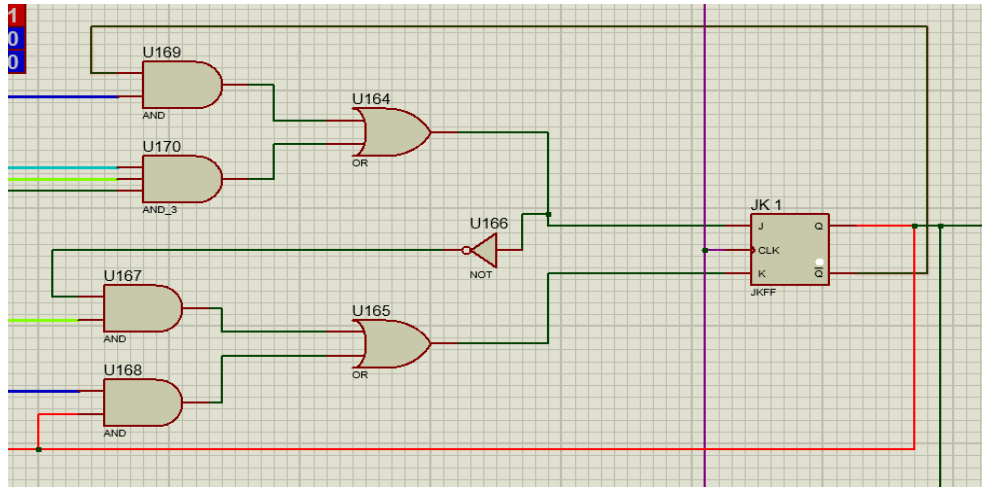


FIG: single JK flip-flop

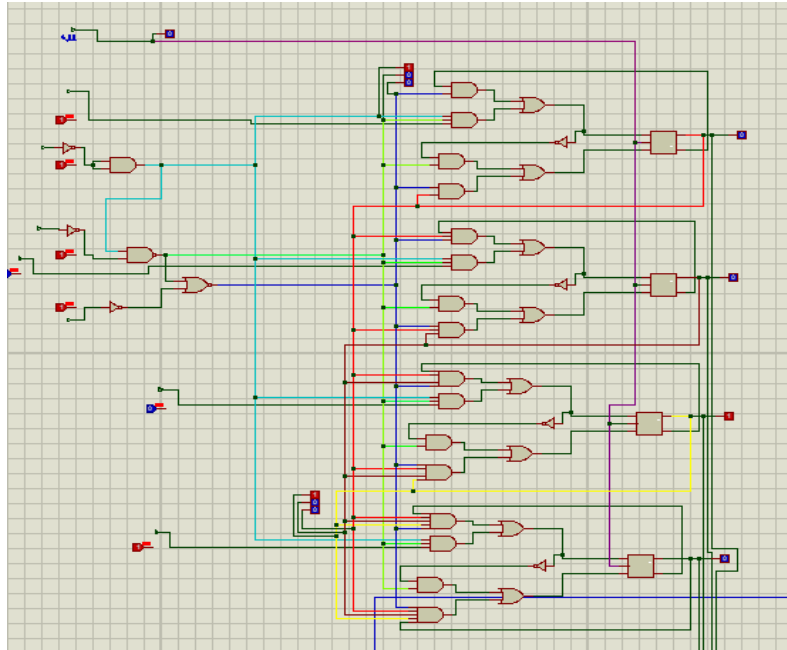


FIG: All four JK flip-flops connected together

1. Architectural Changes

- **Synchronous Operation:** Unlike the original ripple counter, all flip-flops in the 74LS163A are triggered simultaneously by the master clock edge. This eliminates "glitch" states during transitions and ensures timing alignment with the rest of the SAP-1 control unit.
- **Bus Integration:** The counter's parallel data inputs (D1,D2,D3,D4) are interfaced with the instruction register/bus. This allows the jump target address to be "jam-loaded" into the counter during the execution phase of the JUMP instruction.
- **Control Logic Integration:** A new control signal, **Jump (LP)**, was mapped to the counter's **LOAD** pin. When LP is active (low), the counter ignores the increment function and loads the value currently on the bus at the next rising clock edge.

2. Components of the Counter

The new counter module consists of the following functional blocks:

- **Synchronous Clear ($\overline{CLR}$):** A low level on this pin resets all four flip-flop outputs to logic low on the next positive clock edge. This is used for the system **RESET** function.
- **Parallel Load ($\overline{LOAD}$):** When pulled low, this disables the counting sequence and enables the parallel data inputs. This is the primary mechanism for the **JUMP** instruction.
- **Enable Inputs (ENP and ENT):** Both pins must be high for the counter to increment. In the SAP-1, these are tied to the **Count (CP)** signal. **ENT** is also fed forward to the Ripple Carry Output for cascading.
- **JK Flip-Flop Array:** The core storage elements. As seen in the logic diagram, the J and K inputs are driven by a combinational steering logic gate network that determines whether the next state is a reset, a load, or an increment.